

Predicción de incumplimiento crediticio

Autor: Edwin Daniel Proaño Zapata

Carrera de Ingeniera en Sistemas de la Información

Trabajo de Titulación modalidad proyecto practico Complexivo

Quito, 2026

TITULO: Predicción de incumplimiento crediticio

Autor: Edwin Daniel Proaño Zapata

La presente investigación tuvo como objetivo comparar el desempeño de cuatro algoritmos de aprendizaje automático en la predicción del incumplimiento crediticio durante un período de doce meses. Para ello, se trabajó con un conjunto de datos sintéticos de 50.000 registros de solicitantes de crédito y 14 variables predictoras, bajo un enfoque cuantitativo, experimental y comparativo. Los datos se dividieron de forma estratificada en dos grupos: 80 % para el entrenamiento y 20 % para la prueba. Se implementaron los algoritmos de Regresión Logística, Árbol de Decisión, Random Forest y Gradient Boosting, cuyo desempeño se evaluó mediante validación cruzada estratificada de cinco particiones. Debido al desbalance de clases, ya que solo el 5,48 % de los registros correspondía a casos de incumplimiento, se utilizó Average Precision (AP) como principal métrica de comparación.

Los resultados de la validación cruzada mostraron que Gradient Boosting obtuvo el mayor AP promedio, con un valor de 0,113952, por lo que fue seleccionado como el modelo con mejor desempeño. En la evaluación final con el conjunto de prueba (TEST), alcanzó un AP de 0,124515, un ROC-AUC de 0,678975 y una exactitud del 94,52 %. Sin embargo, al utilizar un umbral de clasificación de 0,50, el modelo no logró identificar ninguno de los 548 casos reales de incumplimiento, lo que dio como resultado un Recall del 0 %. El análisis de sensibilidad permitió observar que la reducción del umbral mejoraba la detección de estos casos, aunque también incrementaba considerablemente los falsos positivos. Además, se analizaron los costos hipotéticos asociados a los errores de clasificación y se desarrolló un dashboard interactivo para facilitar la visualización e interpretación de los resultados.

En conclusión, Gradient Boosting presentó el mejor desempeño de acuerdo con la métrica establecida para la selección del modelo; sin embargo, tuvo dificultades para identificar los casos de incumplimiento cuando se utilizó el umbral convencional de 0,50. Estos resultados muestran que una alta exactitud no garantiza que un modelo identifique correctamente los casos de riesgo, especialmente cuando existe un desbalance importante entre las clases. Por ello, la evaluación de modelos de riesgo crediticio debe considerar distintas métricas de desempeño y los criterios de decisión utilizados, con el fin de lograr una identificación más adecuada de los solicitantes con mayor probabilidad de incumplimiento.

Palabras clave: Aprendizaje automatizado, Clasificacion Binaria, Análisis exploratorio de datos, Datos de entrenamiento, Datos de prueba, Preprocesamiento, Rango intercuartílico, One-Hot Encoding, Fuga de información, Regresión Logística, Árbol de Decisión, Random Forest, Gradient Boosting, Validación cruzada estratificada, Predicciones fuera de partición, Matriz de confusión, Exactitud, Precisión, Sensibilidad, Precisión promedio.

TITLE: Credit Default Prediction

Author: Edwin Daniel Proaño Zapata

The objective of this research was to compare the performance of four machine learning algorithms in predicting credit default over a twelve-month period. To achieve this, a synthetic dataset containing 50,000 credit applicant records and 14 predictor variables was used, following a quantitative, experimental, and comparative approach. The data were stratified into two subsets: 80% for training and 20% for testing. Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting algorithms were implemented, and their performance was evaluated using five-fold stratified cross-validation. Due to class imbalance, with only 5.48% of the records corresponding to default cases, Average Precision (AP) was selected as the primary evaluation metric for model comparison.

The cross-validation results showed that Gradient Boosting achieved the highest mean AP of 0.113952 and was therefore selected as the best-performing model according to the established criterion. In the final evaluation on the test set (TEST), the model achieved an AP of 0.124515, a ROC-AUC of 0.678975, and an accuracy of 94.52%. However, when applying a classification threshold of 0.50, the model failed to identify any of the 548 actual default cases, resulting in a recall of 0%. The sensitivity analysis revealed that lowering the classification threshold improved the detection of default cases, although it also considerably increased the number of false positives. Additionally, hypothetical costs associated with classification errors were analyzed, and an interactive dashboard was developed to facilitate the visualization and interpretation of the results.

In conclusion, Gradient Boosting demonstrated the best performance according to the metric established for model selection; however, it experienced difficulties identifying

default cases when the conventional classification threshold of 0.50 was applied. These findings demonstrate that high accuracy does not guarantee that a model can correctly identify high-risk cases, particularly in the presence of substantial class imbalance. Therefore, the evaluation of credit risk models should consider multiple performance metrics and decision-making criteria to achieve more effective identification of applicants with a higher probability of default.

Keywords: Machine Learning (ML), Binary Classification, Exploratory Data Analysis (EDA), Training Set (TRAIN), Test Set (TEST), Preprocessing, Interquartile Range (IQR), One-Hot Encoding (OHE), Data Leakage, Logistic Regression (LR), Decision Tree (DT), Random Forest (RF), Gradient Boosting (GB), Stratified K-Fold, Out-of-Fold (OOF), Confusion Matrix, Accuracy, Precision, Recall, Average Precision (AP).

ÍNDICE DE CONTENIDO

1 CUERPO DEL DOCUMENTO 13

1.1 Introducción 13

1.1.1 Objetivo general 14

1.1.2 Objetivos específicos 14

1.2 Teoría 14

1.2.1 Riesgo e incumplimiento crediticio 14

1.2.2 Aprendizaje automático y clasificación supervisada 15

1.2.3 Desbalance de clases 16

1.2.4 Análisis exploratorio y preprocesamiento de datos 16

1.2.5 Algoritmos de clasificación supervisada 17

1.2.6 Validación cruzada y capacidad de generalización 20

1.2.7 Métricas de evaluación 20

1.2.8 Umbral de clasificación y costos de los errores 22

1.2.9 Visualización y reproducibilidad 23

1.3 Metodología experimental 23

1.3.1 Obtención y descripción del conjunto de datos 25

1.3.2 División de los datos de entrenamiento y prueba 26

1.3.3 Análisis exploratorio de datos 27

1.3.4 Procesamiento de datos 29

1.3.5 Selección de algoritmos de clasificación 30

1.3.6 Tratamiento del desbalance de clases 31

1.3.7 Entrenamiento mediante validación cruzada estratificada 32

1.3.8 Métricas de evaluación 33

1.3.9 Selección del modelo ganador 34

1.3.10 Análisis de sensibilidad del umbral de clasificación 35

1.3.11 Evaluación de costos hipotéticos 36

1.3.12 Entrenamiento definitivo y evaluación sobre TEST 37

1.3.13 Presentación y visualización de resultados 38

1.3.14 Reproducibilidad y consideraciones metodológicas 39

1.4 Análisis y Resultados 39

1.4.1 Obtención y descripción del conjunto de datos 40

1.4.2 División de datos de entrenamiento y prueba 41

1.4.3 Análisis exploratorio de datos 43

1.4.3.1 Resultados de las variables categóricas 44

1.4.3.2 Resultados del análisis de cuartiles y valores atípicos 46

1.4.4 Preprocesamiento de los datos 48

1.4.5 Selección de algoritmos de clasificación 49

1.4.6 Tratamiento del desbalance de clases 51

1.4.7 Entrenamiento mediante validación cruzada estratificada 51

1.4.8 Métricas de evaluación 54

1.4.8.1 Matrices de confusión de TRAIN 56

1.4.8.2 Curvas ROC y Precision–Recall 57

1.4.9 Selección del modelo ganador 58

1.4.10 Sensibilidad del umbral de clasificación 60

1.4.11 Evaluación de costos hipotéticos 62

1.4.12 Entrenamiento definitivo y evaluación sobre TEST 63

1.4.13 Presentación y visualización del desempeño final 66

1.4.14 Reproducibilidad y consideraciones metodológicas 67

1.4.15 Presentación y visualización mediante el dashboard 69

1.5 Discusión de Resultados 70

1.6 Conclusiones 71

1.7 Recomedaciones 72

1.8 Citas Bibliográficas 73

2 ELEMENTOS COMPLEMENTARIOS 75

2.1 Anexos 75

ÍNDICE DE TABLAS

Tabla 1. Algoritmos de clasificación implementados en el experimento 40

Tabla 2. Distribución de registros según clase y conjunto de datos 41

Tabla 3. Estadísticas descriptivas de las principales variables numéricas de TRAIN 43

Tabla 4. Tasa de incumplimiento según tipo de empleo 44

Tabla 5. Variables con valores atípicos en TRAIN 46

Tabla 6. Resultados del preprocesamiento de las variables 48

Tabla 7. Algoritmos de clasificación implementados en el experimento 50

Tabla 8. Distribución de las clases en TRAIN 51

Tabla 9. Average Precision obtenido por modelo en cada fold 52

Tabla 10. Media y desviación estándar de la métrica porcentajes 53

Tabla 11. Comparación de las métricas promedio obtenidas mediante validación cruzada 54

Tabla 12. Comparación de las métricas porcentajes 55

Tabla 13. Resultados de las matrices de confusión OOF 56

Tabla 14. Métricas globales OOF asociadas a las curvas 57

Tabla 15. Clasificación de los algoritmos según su AP promedio 58

Tabla 16. Variación de Precision y Recall según el umbral de clasificación 60

Tabla 17. Métricas globales OOF asociadas a las curvas 62

Tabla 18. Métricas finales de Gradient Boosting en TEST 64

Tabla 19. Matriz de confusión de Gradient Boosting en TEST 64

Tabla 20. Comparación de resultados CV frente a TEST 66

Tabla 21. Configuración reproducible del experimento 68

Tabla 22. Correspondencia de las secciones del dashboard con sus fuentes 69

ÍNDICE DE FIGURAS

Figura 1. Distribución de las clases por conjunto de datos 42

Figura 2. Tasa de incumplimiento por tipo de empleo 45

Figura 3. Proporción de valores atípicos por variable 47

Figura 4. AP de validación por fold 52

Figura 5. Media y desviación estándar de la métrica porcentajes 53

Figura 6. Comparación de las métricas promedio obtenidas mediante validación cruzada porcentajes 55

Figura 7. Comparación de Average Precision promedio 59

Figura 8. Análisis de sensibilidad del umbral 61

Figura 9. Comparación de costos hipotéticos por modelo 63

Figura 10. Matriz de confusión final del modelo ganador 65

Figura 11. Comparación CV frente a TEST en Gradient Boosting 67

1 CUERPO DEL DOCUMENTO

1.1 Introducción

El incumplimiento crediticio representa un riesgo para las instituciones financieras debido a las pérdidas económicas que puede generar. Por ello, identificar anticipadamente a los solicitantes con mayor probabilidad de incumplimiento es importante para evaluar el riesgo. En este contexto, el aprendizaje automático permite analizar información económica, laboral y crediticia para desarrollar modelos predictivos.

Uno de los principales desafíos es el desbalance de clases, debido a que los casos de incumplimiento suelen ser menos frecuentes que los de cumplimiento. Esto puede provocar que un modelo alcance una exactitud elevada sin identificar adecuadamente a los solicitantes de mayor riesgo. Por esta razón, es necesario utilizar métricas que permitan evaluar su capacidad para detectar estos casos.

Esta investigación compara cuatro algoritmos de aprendizaje automático: Regresión Logística, Árbol de Decisión, Random Forest y Gradient Boosting, para predecir el incumplimiento crediticio en un período de doce meses. Se utiliza un conjunto de datos sintéticos de 50.000 registros y Average Precision (AP) como métrica principal para seleccionar el modelo con mejor desempeño.

Además, se analizan diferentes umbrales de clasificación y los costos hipotéticos asociados a los errores de predicción. Los resultados se presentan mediante un dashboard interactivo que facilita la comparación de los modelos. Debido al uso de datos sintéticos, las conclusiones se limitan al ámbito experimental y no constituyen una validación para su aplicación directa en instituciones financieras reales.

1.1.1 Objetivo general

Comparar el desempeño de cuatro algoritmos de aprendizaje automático para predecir el incumplimiento crediticio a doce meses, utilizando datos sintéticos y métricas adecuadas para problemas de clasificación con desbalance de clases.

1.1.2 Objetivos específicos

- Implementar los cuatro algoritmos seleccionados para predecir el incumplimiento crediticio a partir de las características de los solicitantes.
- Evaluar y comparar los modelos mediante validación cruzada y métricas de clasificación, utilizando Average Precision (AP) como criterio principal.
- Analizar el efecto de diferentes umbrales de clasificación en la detección de incumplimientos y la generación de falsos positivos.
- Examinar los costos hipotéticos de los errores de clasificación y sus posibles implicaciones en la evaluación del riesgo crediticio.
- Desarrollar un dashboard interactivo para visualizar los resultados y comparar el desempeño de los modelos.

1.2 Teoría

1.2.1 Riesgo e incumplimiento crediticio

El riesgo crediticio es la posibilidad de que una persona o entidad no cumpla con sus obligaciones financieras dentro de los plazos establecidos. Su evaluación permite a las

instituciones financieras identificar el nivel de riesgo de los solicitantes y tomar decisiones más informadas al otorgar créditos.

El incumplimiento crediticio, conocido como default, ocurre cuando un deudor no realiza los pagos según las condiciones acordadas. Para analizarlo se consideran factores como los ingresos, el endeudamiento, la estabilidad laboral y el historial de pagos, que ayudan a identificar posibles dificultades financieras.

Según Hand y Henley (1997), los métodos de clasificación estadística permiten evaluar el riesgo crediticio a partir de las características y el comportamiento financiero de los solicitantes. Estos métodos sirven como base para desarrollar modelos que distinguen diferentes perfiles de riesgo.

1.2.2 Aprendizaje automático y clasificación supervisada

El aprendizaje automático (Machine Learning) es una rama de la inteligencia artificial que permite identificar patrones en los datos y utilizarlos para realizar predicciones. Sus aplicaciones incluyen la clasificación de registros, la estimación de valores y el análisis de relaciones entre variables.

El aprendizaje supervisado utiliza datos previamente etiquetados para entrenar modelos predictivos. Durante este proceso, los algoritmos identifican relaciones entre las variables de entrada y una variable objetivo, con el propósito de realizar predicciones sobre nuevos registros.

Uno de sus principales problemas es la clasificación binaria, que consiste en asignar cada observación a una de dos categorías. En el ámbito crediticio, permite distinguir entre clientes que cumplen con sus obligaciones y aquellos que presentan incumplimiento.

Pedregosa et al. (2011) describen Scikit-learn como una biblioteca de Python que integra algoritmos de aprendizaje automático y facilita el procesamiento de datos, el entrenamiento y la evaluación de modelos predictivos.

1.2.3 Desbalance de clases

El desbalance de clases ocurre cuando una categoría contiene una cantidad considerablemente mayor de registros que otra. En el análisis crediticio, los clientes que cumplen con sus pagos suelen representar la mayoría, mientras que los casos de incumplimiento tienen menor presencia.

Esta diferencia puede afectar el rendimiento de los algoritmos, ya que tienden a favorecer la clase mayoritaria. Como resultado, un modelo puede alcanzar una exactitud elevada sin identificar correctamente los casos de incumplimiento.

Saito y Rehmsmeier (2015) destacan que las curvas Precision–Recall son especialmente útiles cuando existe desbalance de clases, debido a que permiten evaluar la identificación de los casos positivos y la precisión de las predicciones.

Para abordar este problema se utilizan técnicas como la ponderación de clases, el sobremuestreo y el submuestreo. Estas permiten ajustar la importancia de los errores o modificar la distribución de los datos de entrenamiento. Su aplicación depende de las características del conjunto de datos y del algoritmo utilizado.

1.2.4 Análisis exploratorio y preprocesamiento de datos

El análisis exploratorio de datos (EDA) permite conocer las características de un conjunto de información mediante procedimientos estadísticos y gráficos. Su objetivo es

identificar distribuciones, relaciones entre variables, valores atípicos y posibles inconsistencias antes de construir un modelo predictivo.

Entre las medidas más utilizadas están la media, mediana, desviación estándar y cuartiles, que permiten analizar el comportamiento de las variables numéricas. También se emplea el rango intercuartílico (IQR) para identificar posibles valores atípicos.

El preprocesamiento consiste en preparar los datos para su utilización en los algoritmos de aprendizaje automático. Incluye el tratamiento de valores faltantes, la codificación de variables categóricas y la estandarización de variables numéricas.

Entre sus técnicas se encuentran One-Hot Encoding, que transforma las categorías en variables binarias, y StandardScaler, que estandariza los valores numéricos utilizando una media de cero y una desviación estándar de uno.

Scikit-learn developers (s. f.) destaca la importancia de aplicar correctamente estas transformaciones para evitar la fuga de información (data leakage), que ocurre cuando se utilizan datos de validación o prueba durante el entrenamiento y puede generar evaluaciones demasiado optimistas.

1.2.5 Algoritmos de clasificación supervisada

Los algoritmos de clasificación supervisada utilizan datos etiquetados para aprender patrones y asignar nuevas observaciones a una categoría. Aunque tienen el mismo propósito, emplean diferentes procedimientos para construir sus predicciones, por lo que su rendimiento depende de las características de los datos.

- **Regresión Logística**

Es un algoritmo utilizado principalmente en clasificación binaria. Su objetivo es estimar la probabilidad de que una observación pertenezca a una clase a partir de sus variables predictoras. Para ello, utiliza la función logística o sigmoide, que transforma una combinación lineal de variables en un valor entre cero y uno. En el análisis crediticio, este resultado puede representar la probabilidad estimada de incumplimiento de un cliente.

Su principal ventaja es la facilidad de interpretación, ya que sus coeficientes permiten analizar la relación entre las variables y la predicción. Sin embargo, puede presentar limitaciones al representar relaciones complejas entre los datos.

- **Árbol de Decisión:**

Es un algoritmo que clasifica registros mediante una estructura formada por nodos y ramas. Su funcionamiento consiste en dividir progresivamente los datos según las características que mejor diferencian las categorías, hasta llegar a las hojas donde se establece la predicción final.

Su principal ventaja es que permite interpretar fácilmente las reglas de clasificación. No obstante, cuando alcanza una complejidad excesiva, puede presentar sobreajuste, es decir, aprender características particulares de los datos de entrenamiento que no se mantienen en nuevas observaciones.

- **Random Forest:**

Es un algoritmo de ensamble que combina múltiples árboles de decisión para generar una predicción conjunta. Cada árbol se entrena utilizando muestras de los datos y subconjuntos de características, lo que permite obtener diferentes resultados que posteriormente se combinan.

Breiman (2001) explica que la incorporación de elementos aleatorios permite desarrollar clasificadores cuyo rendimiento depende de la capacidad predictiva de los árboles y de la relación existente entre ellos.

Este enfoque puede reducir la variabilidad de un árbol individual y permite identificar relaciones complejas entre las variables, mejorando la estabilidad de las predicciones.

- **Gradient Boosting:**

Es un método de ensamble que construye modelos de forma secuencial. A diferencia de Random Forest, incorpora nuevos modelos progresivamente para reducir los errores de las predicciones anteriores.

Friedman (2001) desarrolló este enfoque mediante la optimización de funciones de pérdida, combinando modelos simples para construir un predictor más complejo.

Este algoritmo permite identificar relaciones no lineales e interacciones entre variables. Su rendimiento depende de parámetros como la complejidad de los árboles, la tasa de aprendizaje y el número de iteraciones. Una de sus implementaciones es

HistGradientBoostingClassifier de Scikit-learn, que utiliza histogramas para realizar divisiones de manera eficiente.

1.2.6 Validación cruzada y capacidad de generalización

La evaluación de un modelo requiere analizar su comportamiento con datos que no hayan participado en el entrenamiento. Esto permite conocer su capacidad de generalización, es decir, qué tan bien puede realizar predicciones sobre nuevas observaciones.

Una práctica habitual consiste en dividir los datos en entrenamiento (TRAIN) y prueba (TEST). El primero se utiliza para desarrollar los modelos, mientras que el segundo permite evaluar su rendimiento final de manera independiente.

La validación cruzada (Cross-Validation) complementa este proceso al dividir los datos de entrenamiento en varios subconjuntos o folds. En cada iteración, una parte se utiliza para validar el modelo y las restantes para entrenarlo.

La validación cruzada estratificada mantiene aproximadamente la proporción de las clases en cada partición. Según Scikit-learn developers (s. f.), StratifiedKFold permite realizar estas divisiones, especialmente útiles cuando existe desbalance de clases.

Por otra parte, las predicciones Out-of-Fold (OOF) se obtienen sobre registros que no participaron en el entrenamiento del modelo que los predice. Esto permite evaluar el comportamiento de los clasificadores sin utilizar el conjunto de prueba final.

1.2.7 Métricas de evaluación

Las métricas de evaluación permiten medir el rendimiento de los modelos y determinar qué tan correctamente clasifican los registros. En clasificación binaria se utiliza la

matriz de confusión, que organiza los resultados en verdaderos positivos (VP), verdaderos negativos (VN), falsos positivos (FP) y falsos negativos (FN).

Los verdaderos positivos y negativos representan clasificaciones correctas, mientras que los falsos positivos y negativos corresponden a errores. A partir de estos resultados se calculan las siguientes métricas:

- **Accuracy (exactitud):** mide la proporción de predicciones correctas. Puede ser poco representativa cuando existe desbalance de clases.

- **Precision (precisión):** indica qué proporción de los registros clasificados como positivos realmente pertenece a esa categoría.

- **Recall (sensibilidad):** mide la capacidad del modelo para identificar correctamente los casos positivos.

- **F1-Score:** combina Precision y Recall mediante su media armónica, permitiendo evaluar el equilibrio entre ambas.

- **ROC-AUC**: evalúa la capacidad del modelo para distinguir entre las clases considerando diferentes umbrales de clasificación.

- **Average Precision (AP):** resume el comportamiento de Precision en función de los cambios de Recall.

Average Precision es especialmente útil cuando existe desbalance de clases. Saito y Rehmsmeier (2015) destacan la importancia de las curvas Precision–Recall para evaluar modelos cuando los casos positivos representan una proporción reducida.

Según Scikit-learn developers (s. f.), la función average_precision_score calcula AP mediante una suma ponderada de los valores de Precision, utilizando los incrementos de Recall como pesos, por lo que no equivale necesariamente al área calculada mediante integración trapezoidal.

La interpretación conjunta de estas métricas permite identificar las fortalezas y limitaciones de cada modelo, especialmente cuando se busca detectar correctamente los casos de incumplimiento crediticio.

1.2.8 Umbral de clasificación y costos de los errores

Los modelos de clasificación probabilística generan valores que representan la probabilidad estimada de pertenecer a una clase. Para convertir estos resultados en categorías es necesario establecer un umbral de decisión.

En clasificación binaria se utiliza frecuentemente un umbral de 0,50, aunque no siempre es el más adecuado. Reducirlo puede aumentar la detección de casos positivos, pero también incrementar los falsos positivos.

El análisis de sensibilidad del umbral permite evaluar cómo cambian las predicciones al modificar este valor, especialmente cuando los errores tienen consecuencias diferentes.

En el riesgo crediticio, un falso negativo ocurre cuando un cliente incumplidor es clasificado como cumplidor, mientras que un falso positivo se presenta cuando un cliente que cumple con sus pagos es identificado como incumplidor.

Ambos errores pueden generar consecuencias económicas y afectar las decisiones de crédito. Según Scikit-learn developers (s. f.), el ajuste del umbral puede considerar criterios

de utilidad y costos asociados a cada error, permitiendo seleccionar una clasificación más adecuada para el problema estudiado.

1.2.9 Visualización y reproducibilidad

La visualización de resultados facilita la interpretación del comportamiento de los modelos predictivos. Mediante gráficos, tablas y representaciones estadísticas es posible comparar algoritmos, analizar métricas e identificar diferencias en su rendimiento.

Los paneles interactivos o dashboards permiten reunir esta información en una interfaz organizada, facilitando la consulta de resultados y su interpretación.

Por otra parte, la reproducibilidad consiste en documentar las condiciones necesarias para repetir un experimento y obtener resultados consistentes. Esto incluye registrar los datos utilizados, las transformaciones, los parámetros de los algoritmos y el control de los procesos aleatorios.

Pedregosa et al. (2011) destacan que Scikit-learn facilita la implementación de procesos de aprendizaje automático mediante herramientas que mantienen una estructura de trabajo consistente.

En conjunto, la visualización y la reproducibilidad permiten presentar resultados más claros, verificar los procedimientos aplicados y respaldar las conclusiones obtenidas durante la evaluación de los modelos.

1.3 Metodología experimental

La presente investigación adoptó un enfoque cuantitativo, experimental y comparativo, con el objetivo de evaluar el desempeño de distintos algoritmos de aprendizaje

automático en la predicción del incumplimiento crediticio. Para ello, se utilizó el conjunto de datos sintéticos proporcionado para el Caso 1, a fin de identificar patrones relacionados con el riesgo de incumplimiento en un horizonte de doce meses.

El desarrollo metodológico comprendió varias etapas:

- Obtención e inspección de los datos
- Análisis exploratorio
- Preprocesamiento de las variables
- La división del conjunto de datos.

Posteriormente, se entrenaron cuatro algoritmos de clasificación supervisada y se evaluó su desempeño mediante validación cruzada estratificada. A partir de los resultados obtenidos, se seleccionó el modelo con mejor rendimiento y se realizó una evaluación final utilizando datos independientes.

La implementación se llevó a cabo en Python, utilizando las bibliotecas Pandas, NumPy y Scikit-learn. Para favorecer la reproducibilidad del experimento, se estableció una semilla aleatoria fija de 42 (random_state=42), utilizada en la división de los datos y en la validación cruzada estratificada. Este valor fue seleccionado de manera convencional, sin representar una ventaja específica en el desempeño de los algoritmos. Su utilización permitió mantener constantes las particiones de los datos entre ejecuciones realizadas bajo las mismas condiciones y garantizar una comparación consistente de los cuatro modelos evaluados.

1.3.1 Obtención y descripción del conjunto de datos

Se utilizó el archivo riesgo_crediticio_bigdata.csv, compuesto por 50.000 registros y 16 columnas, correspondientes a información sobre solicitantes de crédito. Cada registro representa una observación individual que contiene características económicas, laborales y crediticias.

Se identificaron 14 variables predictoras, una variable objetivo y un identificador único del solicitante. Las variables predictoras comprenden edad, ingresos mensuales, años de empleo, tipo de empleo, situación de vivienda, monto solicitado, plazo del crédito, relación deuda-ingresos, utilización del crédito, pagos atrasados, créditos abiertos, ahorros, finalidad del préstamo y existencia de un garante.

La variable objetivo, denominada default_12m, representa la condición de incumplimiento crediticio durante los doce meses siguientes y presenta dos categorías:

- **Clase 0:** solicitantes que no registran incumplimiento crediticio.
- **Clase 1:** solicitantes que registran incumplimiento crediticio.

La columna applicant_id se excluyó del entrenamiento debido a que constituye un identificador y no una característica predictiva del riesgo crediticio.

La inspección inicial permitió confirmar la existencia de 50.000 registros válidos, sin valores faltantes, sin registros exactamente duplicados y sin identificadores repetidos.

1.3.2 División de los datos de entrenamiento y prueba

Antes de realizar el análisis exploratorio supervisado y entrenar los algoritmos de clasificación, se dividió el conjunto de datos en dos subconjuntos independientes mediante la función train_test_split de la biblioteca Scikit-learn.

Para ello, se estableció una proporción del 80 % de los datos para el entrenamiento (TRAIN) y el 20 % restante para la prueba (TEST). Además, se empleó un muestreo estratificado mediante el parámetro stratify=y y se fijó la semilla de aleatoriedad en random_state=42, con el propósito de garantizar la reproducibilidad de los resultados.

La estratificación permitió conservar aproximadamente la misma distribución de las clases de la variable objetivo en ambos subconjuntos, manteniendo la proporción de solicitantes que cumplen con sus obligaciones crediticias y aquellos que presentan incumplimiento. De esta manera, se obtuvo una partición adecuada para entrenar los modelos y evaluar su capacidad de generalización con datos no utilizados durante el entrenamiento.

El conjunto de entrenamiento (TRAIN) se destinó al análisis exploratorio de los datos, la validación cruzada, el entrenamiento de los algoritmos y la selección del modelo con mejor desempeño. Por otro lado, el conjunto de prueba (TEST) se mantuvo aislado durante estas etapas y se utilizó exclusivamente para evaluar el rendimiento final del modelo seleccionado sobre datos no vistos durante el entrenamiento.

Esta estrategia permitió reducir el riesgo de fuga de información (data leakage), que se produce cuando información procedente del conjunto de prueba influye, directa o indirectamente, en el proceso de desarrollo del modelo. En este sentido, Scikit-learn (s. f.) destaca la importancia de separar adecuadamente los datos de entrenamiento y prueba para

obtener una evaluación más fiable del desempeño del modelo y evitar estimaciones excesivamente optimistas de su capacidad de generalización.

1.3.3 Análisis exploratorio de datos

El análisis exploratorio de datos EDA (Exploratory Data Analysis) se llevó a cabo únicamente sobre el conjunto de entrenamiento (TRAIN), con el objetivo de conocer el comportamiento de las variables, analizar sus distribuciones e identificar posibles inconsistencias antes de iniciar el entrenamiento de los modelos.

En el caso de las variables numéricas, se calcularon estadísticas descriptivas como:

- Valores mínimos
- Valores máximos
- Media
- Mediana
- Desviación estándar
- Cuartiles.

Estos resultados permitieron examinar la tendencia central y la dispersión de los datos, así como identificar posibles valores extremos.

Para detectar valores atípicos, se utilizó el rango intercuartílico (IQR). Se consideraron posibles valores atípicos aquellas observaciones:

- Inferiores a: $Q1 - 1{,}5 \times IQR$
- Superiores: $Q3 + 1{,}5 \times IQR$

Por su parte, en las variables categóricas se analizaron las frecuencias de cada categoría y sus respectivas tasas de incumplimiento. Esto permitió explorar las diferencias entre los distintos grupos de solicitantes y observar posibles patrones relacionados con el riesgo crediticio.

Asimismo, se examinó la distribución de la variable objetivo, en la que se observó un marcado desbalance entre las clases. Del total de registros originales, el 5,484 % correspondía a clientes con incumplimiento crediticio, mientras que el 94,516 % restante correspondía a clientes que no presentaron incumplimientos.

Este desbalance fue un aspecto relevante para el desarrollo de los modelos, ya que una alta exactitud (accuracy) no implica necesariamente una buena capacidad para identificar a los clientes que presentan un mayor riesgo de incumplimiento. Por ello, fue necesario considerar métricas de evaluación que permitieran valorar de manera más precisa el desempeño de los modelos en la detección de la clase minoritaria.

Este desbalance influyó en la evaluación de los modelos, ya que una alta exactitud no garantiza una identificación adecuada de los clientes con mayor riesgo de incumplimiento crediticio.

1.3.4 Procesamiento de datos

El preprocesamiento de los datos se realizó mediante las clases ColumnTransformer y Pipeline de Scikit-learn, lo que permitió organizar las transformaciones de las variables e integrarlas directamente en el entrenamiento de cada modelo.

Para las variables numéricas se estableció la imputación mediante la mediana, mientras que en las variables categóricas se utilizó la categoría más frecuente. Si bien el conjunto de datos no contenía valores faltantes, se incluyeron estos procedimientos para mantener un flujo de trabajo estructurado y reproducible.

Las variables categóricas se transformaron mediante la técnica One-Hot Encoding, que convierte cada categoría en una representación numérica mediante variables binarias. Para ello, se utilizó OneHotEncoder con el parámetro handle_unknown='ignore', lo que permitió manejar categorías que no estuvieran presentes durante el ajuste del transformador.

En el caso de la Regresión Logística, se aplicó StandardScaler para estandarizar las variables numéricas y evitar que las diferencias en sus escalas afectaran el proceso de optimización del modelo. En cambio, los algoritmos basados en árboles se entrenaron sin necesidad de esta transformación.

Finalmente, todas las transformaciones se ajustaron únicamente con los datos de entrenamiento correspondientes a cada partición (fold) de la validación cruzada. De esta manera, se evitó utilizar información de los datos de validación durante el preprocesamiento, reduciendo el riesgo de fuga de información (data leakage) y garantizando una evaluación más confiable del desempeño de los modelos.

1.3.5 Selección de algoritmos de clasificación

Para comparar distintas estrategias de clasificación binaria, se implementaron cuatro algoritmos de aprendizaje automático supervisado: Regresión Logística, Árbol de Decisión, Random Forest y Gradient Boosting. Cada modelo se configuró con parámetros específicos, de acuerdo con sus características y funcionamiento.

- **Regresión Logística:** Este modelo estima la probabilidad de que una observación pertenezca a una determinada clase mediante una función logística. Se estableció un máximo de 1.300 iteraciones para facilitar la convergencia del algoritmo y se utilizó el parámetro class_weight='balanced' para compensar el desbalance entre las clases.

- **Árbol de Decisión:** Este algoritmo clasifica las observaciones mediante divisiones sucesivas de los datos según sus características. Para controlar la complejidad del modelo y reducir el riesgo de sobreajuste, se estableció una profundidad máxima de seis niveles y un mínimo de 25 muestras por hoja. Asimismo, se aplicó una ponderación equilibrada de las clases.

- **Random Forest:** Este método de ensamble combina las predicciones de múltiples árboles de decisión para obtener un resultado conjunto. Se configuró con 110 árboles, un mínimo de cinco muestras por hoja y el criterio max_features='sqrt', que limita las características consideradas en cada división. Además, se utilizó class_weight='balanced_subsample' para ajustar los pesos de las clases en cada muestra utilizada durante el entrenamiento de los árboles.

- **Gradient Boosting:** Para este algoritmo se empleó HistGradientBoostingClassifier, que construye un modelo de manera progresiva con el propósito de mejorar las

predicciones a partir de los errores cometidos en las etapas anteriores. La configuración incluyó un máximo de 110 iteraciones, un límite de 15 nodos hoja por árbol, una tasa de aprendizaje de 0,07 y una regularización L2 de 0,1.

Los hiperparámetros de los cuatro algoritmos se definieron antes de realizar la evaluación final y se mantuvieron establecidos durante la comparación. No se efectuó una búsqueda exhaustiva de configuraciones mediante técnicas como GridSearchCV o RandomizedSearchCV, por lo que el análisis se centró en comparar el desempeño de los modelos bajo los parámetros seleccionados.

1.3.6 Tratamiento del desbalance de clases

El conjunto de datos presentó un desbalance considerable entre las clases, ya que el número de solicitantes que no registraron incumplimientos fue significativamente superior al de aquellos que sí incumplieron sus obligaciones crediticias.

Para mantener la proporción de ambas clases durante el desarrollo del experimento, se utilizó una división estratificada tanto en la separación de los conjuntos TRAIN y TEST como en la validación cruzada. De esta manera, se procuró conservar una representación similar de cada clase en las distintas particiones de los datos.

Además, se incorporaron estrategias de ponderación de clases en los modelos de Regresión Logística, Árbol de Decisión y Random Forest, con el fin de otorgar mayor importancia a la clase minoritaria durante el entrenamiento y reducir el efecto del desbalance sobre las predicciones. En cambio, el modelo Gradient Boosting se entrenó sin aplicar una ponderación explícita de las clases.

Por otra parte, no se utilizaron técnicas de sobremuestreo, como SMOTE, ni métodos de submuestreo para modificar la distribución de las clases. Por tanto, se mantuvo la proporción original de los registros, lo que permitió comparar el comportamiento de los algoritmos sin alterar artificialmente la composición del conjunto de datos.

1.3.7 Entrenamiento mediante validación cruzada estratificada

Para comparar el desempeño de los cuatro algoritmos, se utilizó una validación cruzada estratificada de cinco particiones mediante StratifiedKFold, configurada con los parámetros n_splits=5, shuffle=True y random_state=42.

Este procedimiento permitió dividir los 40.000 registros del conjunto TRAIN en cinco subconjuntos de aproximadamente 8.000 observaciones cada uno, manteniendo una proporción similar de las clases en cada partición.

En cada iteración, el modelo se entrenó con cuatro de los subconjuntos, equivalentes a 32.000 registros, y se evaluó con el subconjunto restante, compuesto por aproximadamente 8.000 observaciones. Este proceso se repitió cinco veces por cada algoritmo, de modo que todos los registros de TRAIN se utilizaron una vez como datos de validación. En total, se realizaron 20 entrenamientos y evaluaciones, correspondientes a cinco particiones por cada uno de los cuatro modelos.

A partir de los resultados de cada partición, se calcularon la media y la desviación estándar de las métricas de evaluación. La media permitió estimar el rendimiento promedio de cada algoritmo, mientras que la desviación estándar sirvió para analizar la variabilidad de sus resultados entre las distintas particiones.

Asimismo, se almacenaron las probabilidades predichas para las observaciones de cada subconjunto de validación. Estas predicciones, conocidas como Out-of-Fold (OOF), se utilizaron para construir matrices de confusión, analizar curvas de evaluación y estudiar distintos escenarios de sensibilidad. De esta manera, fue posible examinar el comportamiento de los modelos a partir de predicciones obtenidas sobre datos que no habían intervenido en su entrenamiento, sin utilizar el conjunto TEST durante esta etapa.

1.3.8 Métricas de evaluación

El desempeño de los algoritmos se evaluó mediante seis métricas de clasificación: Precision, Recall, F1-Score, Accuracy, ROC-AUC y Average Precision (AP). Cada una permitió analizar distintos aspectos del rendimiento de los modelos, con especial atención a su capacidad para identificar correctamente a los clientes con incumplimiento crediticio.

La métrica Precision permitió determinar qué proporción de los clientes clasificados como incumplidores realmente pertenecía a esta categoría. Por su parte, Recall midió la proporción de incumplimientos reales que fueron identificados correctamente por el modelo. Esta última métrica tuvo especial relevancia para la investigación, debido a la necesidad de detectar a los solicitantes con mayor riesgo de incumplimiento.

El F1-Score permitió evaluar el equilibrio entre Precision y Recall mediante su media armónica. En cambio, Accuracy midió la proporción total de predicciones correctas respecto al número de observaciones evaluadas. No obstante, debido al desbalance de clases presente en los datos, esta última métrica no se consideró suficiente para determinar por sí sola la calidad de los modelos.

La métrica ROC-AUC se utilizó para evaluar la capacidad de los algoritmos para distinguir entre clientes con y sin incumplimiento a partir de distintos umbrales de clasificación. Adicionalmente, se calculó Average Precision (AP), seleccionada como la métrica principal para comparar los modelos. Esta medida resume el comportamiento de la precisión y la sensibilidad (Recall) a lo largo de los distintos umbrales de decisión, lo que resulta especialmente útil cuando la clase positiva tiene una representación reducida.

La elección de AP como criterio principal se fundamentó en el marcado desbalance de clases del conjunto de datos. En este contexto, Saito y Rehmsmeier (2015) destacan que las curvas de precisión-sensibilidad (Precision–Recall) pueden ofrecer una visión más informativa del rendimiento de los clasificadores que las curvas ROC cuando se trabaja con conjuntos de datos altamente desbalanceados.

Finalmente, es importante distinguir entre Average Precision y el área calculada mediante integración trapezoidal de la curva Precision–Recall. Aunque ambas medidas están relacionadas, sus procedimientos de cálculo son diferentes. En esta investigación, AP se calculó mediante la función average_precision_score de Scikit-learn, por lo que los resultados corresponden a esta definición específica de la métrica.

1.3.9 Selección del modelo ganador

La selección del modelo final se realizó a partir de los resultados obtenidos en la validación cruzada sobre el conjunto TRAIN, sin utilizar los datos de TEST para tomar esta decisión.

Como criterio principal, se estableció la media de Average Precision (AP) obtenida en los cinco folds. En caso de empate entre modelos, se consideró como criterio secundario la media de Recall.

Tras comparar los cuatro algoritmos, Gradient Boosting alcanzó el mayor AP promedio, con un valor de 0,113952. Le siguieron Random Forest, Regresión Logística y Árbol de Decisión. En función de este resultado, Gradient Boosting fue seleccionado como el modelo ganador del experimento.

Sin embargo, esta elección no significa que el modelo haya obtenido los mejores resultados en todas las métricas evaluadas. En particular, el Recall debe analizarse por separado, ya que la capacidad de ordenar a los solicitantes según su riesgo de incumplimiento no implica necesariamente que se identifiquen correctamente todos los casos positivos al utilizar un umbral de clasificación determinado.

1.3.10 Análisis de sensibilidad del umbral de clasificación

Los cuatro algoritmos generaron probabilidades estimadas de incumplimiento para cada observación. Para convertir estas probabilidades en predicciones binarias, se estableció inicialmente un umbral de clasificación de 0,50. De este modo, las observaciones con una probabilidad igual o superior a dicho valor se clasificaron como incumplimientos, mientras que aquellas con una probabilidad inferior se clasificaron como no incumplimientos.

Este umbral se utilizó como referencia fija y no fue seleccionado mediante un proceso de optimización. Su aplicación permitió establecer un criterio uniforme para interpretar las probabilidades y comparar las predicciones de los modelos.

Además, se realizó un análisis de sensibilidad con umbrales comprendidos entre 0,05 y 0,95, en incrementos de 0,05. Este análisis se llevó a cabo exclusivamente con las predicciones Out-of-Fold (OOF) del conjunto TRAIN, sin recurrir a los datos de TEST.

El objetivo fue examinar cómo la variación del umbral afectaba la clasificación de las observaciones y, en consecuencia, la cantidad de verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos. También se analizaron los cambios en las métricas Precision y Recall, con el fin de identificar las diferencias entre los criterios de clasificación más estrictos y aquellos que favorecen una mayor detección de incumplimientos.

Los umbrales alternativos se utilizaron únicamente con fines exploratorios para comprender el comportamiento del modelo ante distintos criterios de decisión. Por tanto, este análisis no modificó el umbral de referencia de 0,50 establecido para la evaluación final del experimento.

1.3.11 Evaluación de costos hipotéticos

Como complemento a la evaluación del desempeño de los modelos, se realizó un análisis de costos hipotéticos asociados a los errores de clasificación. Para ello, se consideraron dos tipos de errores: los falsos negativos (FN), que corresponden a clientes con incumplimiento crediticio clasificados incorrectamente como cumplidores, y los falsos positivos (FP), que corresponden a clientes sin incumplimiento clasificados erróneamente como incumplidores.

Con el propósito de ilustrar las posibles consecuencias económicas de estas equivocaciones, se asignó un costo hipotético de USD 500 por cada falso negativo y de USD

100 por cada falso positivo. A partir de estos valores, el costo total se calculó mediante la siguiente expresión:

$$\text{Costo total} = (FN \times 500) + (FP \times 100)$$

Los valores asignados se utilizaron únicamente como supuestos académicos para comparar el impacto relativo de ambos tipos de error. Por tanto, no representan pérdidas económicas reales ni estimaciones verificadas de una institución financiera.

El análisis se realizó a partir de las matrices de confusión obtenidas con las predicciones *Out-of-Fold* (OOF) del conjunto TRAIN, sin utilizar las predicciones del conjunto TEST. Finalmente, este análisis se consideró complementario y no alteró el criterio oficial de selección del modelo, que se mantuvo basado en el promedio de *Average Precision* (AP) obtenido durante la validación cruzada.

1.3.12 Entrenamiento definitivo y evaluación sobre TEST

Una vez seleccionado Gradient Boosting como el modelo con mejor desempeño en la validación cruzada, se creó una nueva instancia del algoritmo con los parámetros definidos previamente. Posteriormente, el modelo se entrenó con los 40.000 registros del conjunto TRAIN y se evaluó con los 10.000 registros reservados para TEST.

La evaluación final se realizó una sola vez y únicamente sobre el modelo seleccionado, con el fin de mantener el conjunto TEST independiente del proceso de comparación y selección de algoritmos.

Para esta etapa, se calcularon las métricas de clasificación y la matriz de confusión utilizando el umbral de decisión fijo de 0,50. Estos resultados permitieron examinar la

capacidad del modelo para identificar los casos de incumplimiento y distinguirlos de aquellos en los que no se presentó esta situación.

Finalmente, el desempeño obtenido en TEST se contrastó con los resultados de la validación cruzada, lo que permitió valorar el comportamiento del modelo frente a observaciones que no habían sido utilizadas durante su entrenamiento ni en el proceso de selección.

1.3.13 Presentación y visualización de resultados

Los resultados obtenidos durante el experimento se almacenaron en archivos CSV y JSON, lo que facilitó su consulta, análisis y posterior representación gráfica.

Entre los resultados generados se incluyeron tablas comparativas de métricas, estadísticas descriptivas, resultados de cada fold, matrices de confusión, curvas Precision–Recall y ROC, análisis de distintos umbrales de clasificación, comparaciones de costos hipotéticos y métricas correspondientes a la evaluación final.

Posteriormente, los resultados procesados se integraron en un dashboard interactivo desarrollado con tecnologías web, con el objetivo de facilitar la visualización de los datos y la interpretación del desempeño de los modelos.

El dashboard permite explorar los resultados previamente calculados mediante tablas y gráficos, pero no ejecuta nuevos entrenamientos ni modifica las métricas obtenidas durante el experimento. De esta forma, se mantiene la coherencia entre los análisis realizados en Python y la información presentada visualmente, garantizando la trazabilidad de los resultados.

1.3.14 Reproducibilidad y consideraciones metodológicas

La implementación se realizó siguiendo un proceso que comprende la preparación de los datos, el entrenamiento, la validación y la evaluación de los modelos. El código principal se encuentra en `src/entrenar.py`, mientras que el notebook `Caso_48_Paso_a_Paso.ipynb` presenta paso a paso el desarrollo del experimento.

Para que el procedimiento pueda repetirse bajo las mismas condiciones, se estableció una semilla aleatoria fija, se definieron las configuraciones de los modelos y se aplicó validación cruzada estratificada, manteniendo separados los conjuntos de entrenamiento (TRAIN) y prueba (TEST).

Es importante señalar que el estudio se realizó con datos sintéticos y tiene fines académicos. Por lo tanto, los resultados no pueden utilizarse directamente para tomar decisiones crediticias sobre personas reales. Antes de considerar una aplicación práctica, sería necesario validar los modelos con datos reales, calibrar sus probabilidades y analizar los costos y riesgos que implicaría su uso.

1.4 Análisis y Resultados

En este capítulo se presentan e interpretan los resultados del Caso 1 para comparar el desempeño de los algoritmos evaluados y establecer sus principales diferencias.

El análisis aborda los patrones del conjunto de datos, el rendimiento de los modelos y la evaluación independiente del algoritmo seleccionado. Además, compara los resultados de TRAIN y TEST para valorar su capacidad de generalización, considerando que un desempeño similar en ambos conjuntos no garantiza la correcta detección de incumplimientos con el umbral de clasificación elegido.

1.4.1 Obtención y descripción del conjunto de datos

Para el desarrollo del experimento se utilizó la biblioteca **Scikit-learn**, una herramienta de Python especializada en aprendizaje automático que permite implementar, entrenar y evaluar modelos predictivos de manera estructurada.

Mediante esta biblioteca se implementaron cuatro algoritmos de clasificación supervisada: Regresión Logística, Árbol de Decisión, Random Forest y Gradient Boosting. Su selección permitió comparar diferentes enfoques de aprendizaje automático aplicados a la predicción del incumplimiento crediticio.

Tabla 1. Algoritmos de clasificación implementados en el experimento

Algoritmo	Enfoque de aprendizaje	Característica principal
Regresión Logística	Modelo lineal	Estima la probabilidad de incumplimiento a partir de las características del solicitante.
Árbol de Decisión	Modelo basado en reglas	Realiza clasificaciones mediante decisiones sucesivas sobre las características de los datos.
Random Forest	Modelo de ensamble	Combina múltiples árboles de decisión para generar una predicción conjunta.
Gradient Boosting	Modelo de ensamble	Construye árboles secuencialmente para mejorar progresivamente las predicciones.

Fuente: Elaboración propia a partir de los algoritmos implementados mediante Scikit-learn.

La utilización de Scikit-learn facilitó la integración de los cuatro algoritmos en un mismo entorno de trabajo, permitiendo aplicar procedimientos consistentes de entrenamiento, validación y evaluación. Esto favoreció la comparación de sus resultados bajo las mismas particiones de datos y los criterios de evaluación establecidos.

La implementación de modelos con diferentes mecanismos de aprendizaje permitió examinar distintas alternativas para abordar el problema del riesgo crediticio. Sin embargo, la selección de los algoritmos no determinó anticipadamente su efectividad, por lo que fue necesario evaluar su desempeño mediante validación cruzada y métricas de clasificación.

De esta manera, se estableció una base experimental común para comparar los modelos y determinar posteriormente cuál presentó el mayor rendimiento según el criterio de evaluación definido en la investigación.

1.4.2 División de datos de entrenamiento y prueba

La variable objetivo default_12m presenta dos categorías. La Clase 0 identifica a los solicitantes que no registraron incumplimiento crediticio durante los siguientes doce meses, mientras que la Clase 1 corresponde a aquellos que sí presentaron incumplimiento en ese período. Esta clasificación constituye la base para el entrenamiento de los algoritmos y la evaluación de su capacidad para identificar solicitantes con riesgo crediticio.

La división estratificada permitió distribuir los 50.000 registros originales en dos conjuntos: TRAIN (80 %), utilizado para el entrenamiento y selección de modelos, y TEST (20 %), reservado para la evaluación final.

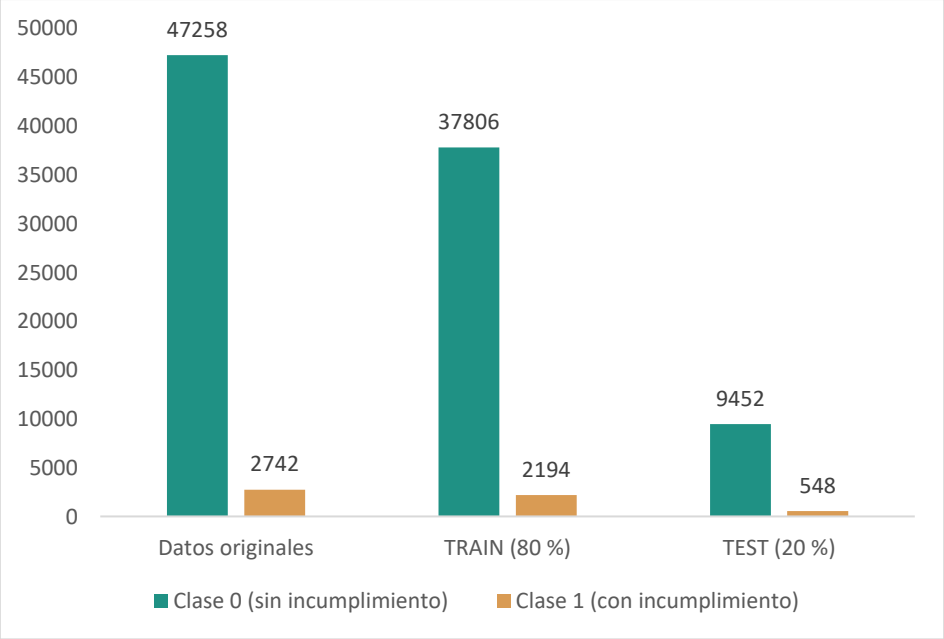
Tabla 2. Distribución de registros según clase y conjunto de datos

Conjunto	Clase 0 (Sin incumplimiento):	Porcentaje (%)	Clase 1 (Con incumplimiento)	Porcentaje (%)	Total
Datos originales	47.258	94,52 %	2.742	5,48 %	50.000
TRAIN (80 %)	37.806	94,52 %	2.194	5,49 %	40.000

Conjunto	Clase 0 (Sin incumplimiento):	Porcentaje (%)	Clase 1 (Con incumplimiento)	Porcentaje (%)	Total
TEST (20 %)	9.452	94,52 %	548	5,48 %	10.000

Fuente: Elaboración propia a partir de resultados_dashboard.json.

Figura 1. Distribución de las clases por conjunto de datos



Fuente: Elaboración propia a partir de resultados_dashboard.json.

Los resultados muestran que el conjunto TRAIN conservó 37.806 solicitantes sin incumplimiento y 2.194 con incumplimiento crediticio. Por su parte, TEST quedó conformado por 9.452 solicitantes de la Clase 0 y 548 de la Clase 1.

La distribución porcentual evidencia que las proporciones se mantuvieron prácticamente constantes después de la división, confirmando el funcionamiento del muestreo estratificado.

Asimismo, se identificó un desbalance considerable entre las clases, dado que aproximadamente el 94,52 % de los registros corresponde a solicitantes sin incumplimiento, mientras que únicamente el 5,48 % representa incumplimientos.

Estos resultados permitieron disponer de un conjunto TRAIN para el entrenamiento y la selección de algoritmos y de un conjunto TEST independiente para evaluar el modelo ganador. La separación contribuyó a prevenir la fuga de información y a obtener una evaluación final sobre registros no utilizados durante el desarrollo de los modelos.

1.4.3 Análisis exploratorio de datos

Para las variables numéricas, se calcularon estadísticas descriptivas como la media, la mediana, la desviación estándar y los valores mínimos y máximos. Estos indicadores permitieron analizar la distribución de los datos, identificar diferencias en sus escalas y observar posibles valores extremos.

Tabla 3. Estadísticas descriptivas de las principales variables numéricas de TRAIN

Variable	Media	Mediana	Desv. estándar	Mínimo	Máximo
Edad (años)	44,08	44,00	15,30	18	70
Ingresos mensuales (USD)	1.221,28	1.051,49	720,57	250	8.000
Antigüedad laboral (años)	11,97	11,00	8,44	0	30
Monto del crédito (USD)	6.624,11	5.043,08	5.523,81	300	40.000
Relación deuda-ingresos	0,3452	0,3280	0,1788	0,010	0,952
Utilización del crédito	0,4662	0,4630	0,2087	0,004	0,993
Pagos atrasados	0,70	1,00	0,84	0	6

Variable	Media	Mediana	Desv. estándar	Mínimo	Máximo
Ahorros (USD)	1.095,86	499,24	1.815,16	0	30.000

Fuente: Elaboración propia a partir de resultados_dashboard.json.

Los resultados evidenciaron que la edad promedio de los solicitantes fue de 44,08 años y que los ingresos mensuales presentaron una media de USD 1.221,28. El monto promedio de los créditos solicitados alcanzó USD 6.624,11.

Se identificaron diferencias entre la media y la mediana de determinadas variables económicas. En particular, los ahorros registraron una media de USD 1.095,86 y una mediana de USD 499,24, lo que sugiere una distribución asimétrica influida por observaciones con valores elevados.

1.4.3.1 Resultados de las variables categóricas

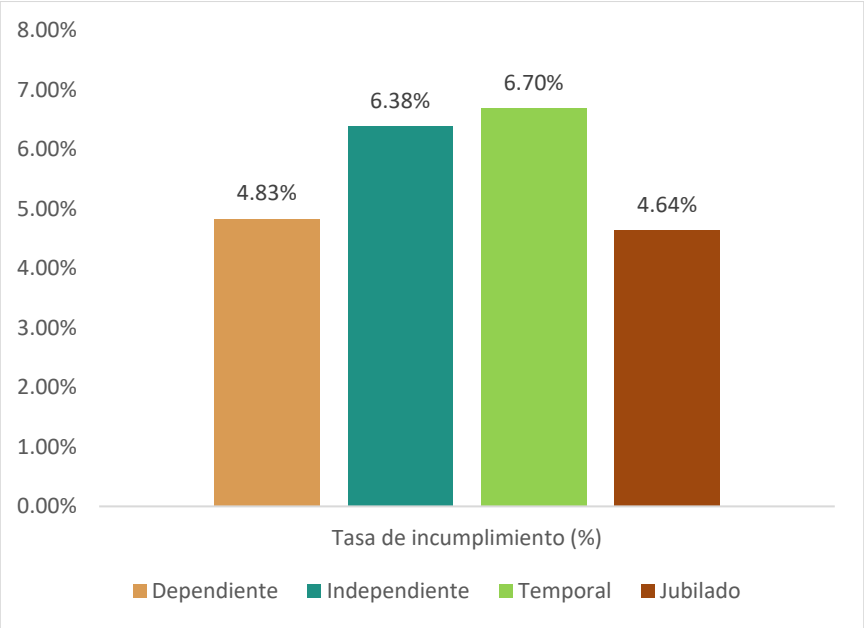
El análisis de las variables categóricas permitió comparar las tasas de incumplimiento entre los distintos grupos de solicitantes e identificar características potencialmente relacionadas con el comportamiento crediticio.

Tabla 4. Tasa de incumplimiento según tipo de empleo

Tipo de empleo	Registros TRAIN	Incumplimiento
Dependiente	21.888	4,83 %
Independiente	11.341	6,38 %
Temporal	4.747	6,70 %
Jubilado	2.024	4,64 %

Fuente: Elaboración propia a partir de resultados_dashboard.json, eda_train.categories.employment_type

Figura 2. Tasa de incumplimiento por tipo de empleo



Fuente: Elaboración propia a partir de resultados_dashboard.json.

El análisis de las variables categóricas permitió identificar diferencias en las tasas de incumplimiento según las características de los solicitantes. En particular, los trabajadores temporales registraron la mayor tasa de incumplimiento, con el 6,70 %, seguidos por los trabajadores independientes, con el 6,38 %. Por su parte, los trabajadores dependientes y jubilados presentaron tasas del 4,83 % y 4,64 %, respectivamente.

Adicionalmente, se observaron diferencias según la existencia de un garante. Los solicitantes sin garante presentaron una tasa de incumplimiento del 5,76 %, mientras que aquellos que contaban con uno registraron una tasa del 4,50 %.

Estos resultados permitieron identificar patrones descriptivos entre los grupos analizados. No obstante, las diferencias observadas no establecen relaciones causales entre las características de los solicitantes y el incumplimiento crediticio.

1.4.3.2 Resultados del análisis de cuartiles y valores atípicos

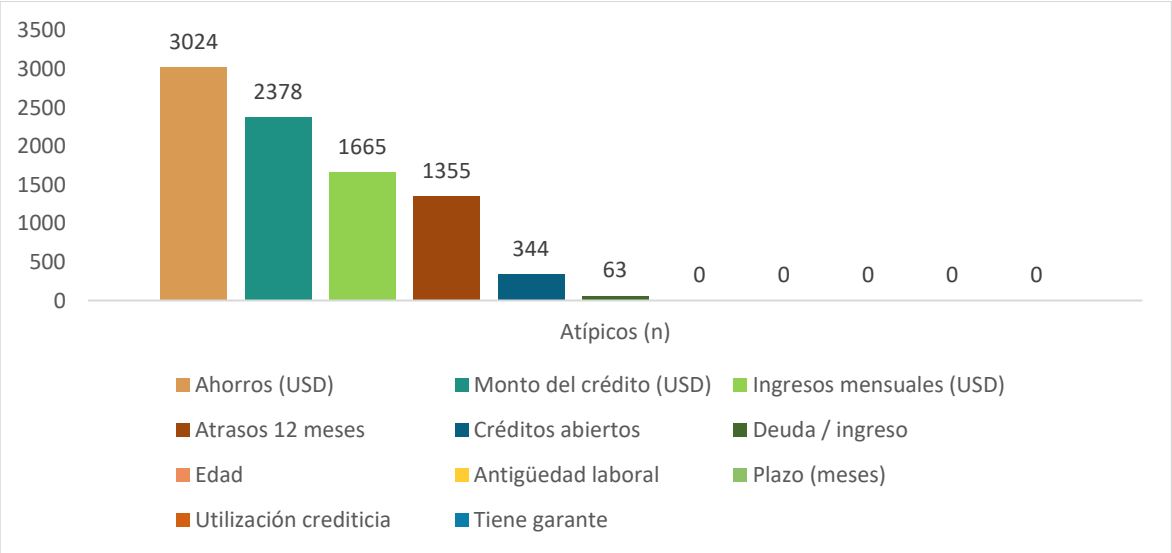
Se identificaron posibles valores atípicos mediante el rango intercuartílico (IQR). Estos se analizaron sin eliminarlos automáticamente, ya que podrían representar situaciones reales dentro del conjunto de datos.

Tabla 5. Variables con valores atípicos en TRAIN

Variable	Q1	Q3	Registros atípicos	% TRAIN
Ahorros (USD)	61,34	1.367,84	3.024	7,56 %
Monto del crédito (USD)	3.027,01	8.302,17	2.378	5,95 %
Ingresos mensuales (USD)	729,34	1.518,77	1.665	4,16 %
Atrasos 12 meses	0	1	1.355	3,39 %
Créditos abiertos	3	5	344	0,86 %
Deuda / ingreso	0,206	0,468	63	0,16 %
Edad	31	57	0	0,00 %
Antigüedad laboral	5	18	0	0,00 %
Plazo (meses)	18	36	0	0,00 %
Utilización crediticia	0,305	0,622	0	0,00 %
Tiene garante	0	0	0	0,00 %

Fuente: Elaboración propia a partir de resultados_dashboard.json, secciones eda_train.numeric y eda_train.outliers

Figura 3. Proporción de valores atípicos por variable



Fuente: Elaboración propia a partir de resultados_dashboard.json.

El análisis de valores atípicos evidenció que la variable ahorros presentó la mayor proporción de observaciones fuera de los límites establecidos mediante el rango intercuartílico, con 3.024 registros (7,56 %), seguida del monto del crédito con 2.378 registros (5,95 %) y los ingresos mensuales con 1.665 registros (4,16 %).

Estas observaciones reflejan la existencia de valores económicos relativamente alejados de la distribución central de los datos. Sin embargo, su identificación estadística no implica necesariamente errores de registro, puesto que pueden representar características legítimas de los solicitantes.

En consecuencia, los valores atípicos fueron considerados durante la caracterización del conjunto de datos, sin proceder a su eliminación automática.

1.4.4 Preprocesamiento de los datos

El preprocesamiento permitió preparar las 14 variables predictoras del conjunto de datos para su utilización en los cuatro algoritmos de clasificación. Se identificaron 11 variables numéricas y 3 categóricas, a las cuales se aplicaron transformaciones específicas mediante ColumnTransformer y Pipeline de Scikit-learn.

La inspección inicial confirmó la ausencia de valores faltantes en los 50.000 registros originales. Por consiguiente, aunque se configuraron procedimientos de imputación mediante mediana y categoría más frecuente, no fue necesario reemplazar valores ausentes. Asimismo, se conservaron los 40.000 registros del conjunto TRAIN.

Tabla 6. Resultados del preprocesamiento de las variables

Característica	Resultado	Transformación aplicada
Variables predictoras originales	14	Identificación de características
Variables numéricas	11	Imputación por mediana
Variables categóricas	3	Imputación por moda y One-Hot Encoding
Variables numéricas de Regresión Logística	11	Estandarización mediante StandardScaler
Valores faltantes detectados	0	No requirieron sustitución
Característica	Resultado	Transformación aplicada
Variables predictoras originales	14	Identificación de características
Variables numéricas	11	Imputación por mediana
Variables categóricas	3	Imputación por moda y One-Hot Encoding
Variables numéricas de Regresión Logística	11	Estandarización mediante StandardScaler

Característica	Resultado	Transformación aplicada
Valores faltantes detectados	0	No requirieron sustitución

Fuente: Elaboración propia a partir de riesgo_crediticio_bigdata.csv, resultados_dashboard.json y la configuración registrada en src/entrenar.py.

Los resultados muestran que el conjunto de datos pudo procesarse sin eliminar observaciones por ausencia de información. Las variables categóricas fueron convertidas a una representación numérica, mientras que las variables numéricas de Regresión Logística fueron estandarizadas para mantener una escala comparable durante el ajuste.

Los modelos basados en árboles conservaron las escalas originales de las variables numéricas. Además, las transformaciones se ajustaron dentro de los respectivos folds de entrenamiento, manteniendo la separación entre los datos utilizados para el ajuste y aquellos destinados a validación.

En consecuencia, el preprocesamiento permitió disponer de una estructura de datos compatible con los algoritmos implementados, conservando la totalidad de los registros TRAIN y reduciendo el riesgo de fuga de información durante la validación cruzada.

1.4.5 Selección de algoritmos de clasificación

Para el desarrollo del experimento se utilizó la biblioteca **Scikit-learn**, una herramienta de Python especializada en aprendizaje automático que permite implementar, entrenar y evaluar modelos predictivos de manera estructurada.

Mediante esta biblioteca se implementaron cuatro algoritmos de clasificación supervisada: Regresión Logística, Árbol de Decisión, Random Forest y Gradient Boosting. Su

selección permitió comparar diferentes enfoques de aprendizaje automático aplicados a la predicción del incumplimiento crediticio.

Tabla 7. Algoritmos de clasificación implementados en el experimento

Algoritmo	Enfoque de aprendizaje	Característica principal
Regresión Logística	Modelo lineal	Estima la probabilidad de incumplimiento a partir de las características del solicitante.
Árbol de Decisión	Modelo basado en reglas	Realiza clasificaciones mediante decisiones sucesivas sobre las características de los datos.
Random Forest	Modelo de ensamble	Combina múltiples árboles de decisión para generar una predicción conjunta.
Gradient Boosting	Modelo de ensamble	Construye árboles secuencialmente para mejorar progresivamente las predicciones.

Fuente: Elaboración propia a partir de los algoritmos implementados mediante Scikit-learn.

La utilización de Scikit-learn facilitó la integración de los cuatro algoritmos en un mismo entorno de trabajo, permitiendo aplicar procedimientos consistentes de entrenamiento, validación y evaluación. Esto favoreció la comparación de sus resultados bajo las mismas particiones de datos y los criterios de evaluación establecidos.

La implementación de modelos con diferentes mecanismos de aprendizaje permitió examinar distintas alternativas para abordar el problema del riesgo crediticio. Sin embargo, la selección de los algoritmos no determinó anticipadamente su efectividad, por lo que fue necesario evaluar su desempeño mediante validación cruzada y métricas de clasificación.

De esta manera, se estableció una base experimental común para comparar los modelos y determinar posteriormente cuál presentó el mayor rendimiento según el criterio de evaluación definido en la investigación.

1.4.6 Tratamiento del desbalance de clases

El análisis confirmó un desbalance considerable en la variable objetivo. En el conjunto TRAIN, 37.806 registros correspondían a solicitantes sin incumplimiento (Clase 0), mientras que solo 2.194 correspondían a clientes con incumplimiento crediticio (Clase 1).

Tabla 8. Distribución de las clases en TRAIN

Clase	Significado	Registros	Porcentaje
0	Sin incumplimiento	37.806	94,515 %
1	Con incumplimiento	2.194	5,485 %
Total		40.000	100 %

Fuente: Elaboración propia a partir de resultados_dashboard.json.

La Clase 0 presentó aproximadamente 17,23 veces más observaciones que la Clase 1, lo que pudo favorecer la predicción de la clase mayoritaria. De hecho, clasificar todos los registros de TRAIN como Clase 0 habría generado una Accuracy del 94,515 %, sin detectar ningún incumplimiento. Para afrontar este desbalance, se utilizó una división estratificada y se aplicaron ponderaciones de clase en Regresión Logística, Árbol de Decisión y Random Forest. No se emplearon técnicas de sobremuestreo como SMOTE, por lo que el desequilibrio se mantuvo como un factor relevante en la evaluación de los cuatro modelos.

1.4.7 Entrenamiento mediante validación cruzada estratificada

La validación cruzada estratificada de cinco particiones permitió obtener y comparar los resultados de los cuatro algoritmos de clasificación sobre el conjunto TRAIN, conformado por 40.000 registros. En total, se realizaron 20 evaluaciones, correspondientes a cinco folds por cada algoritmo.

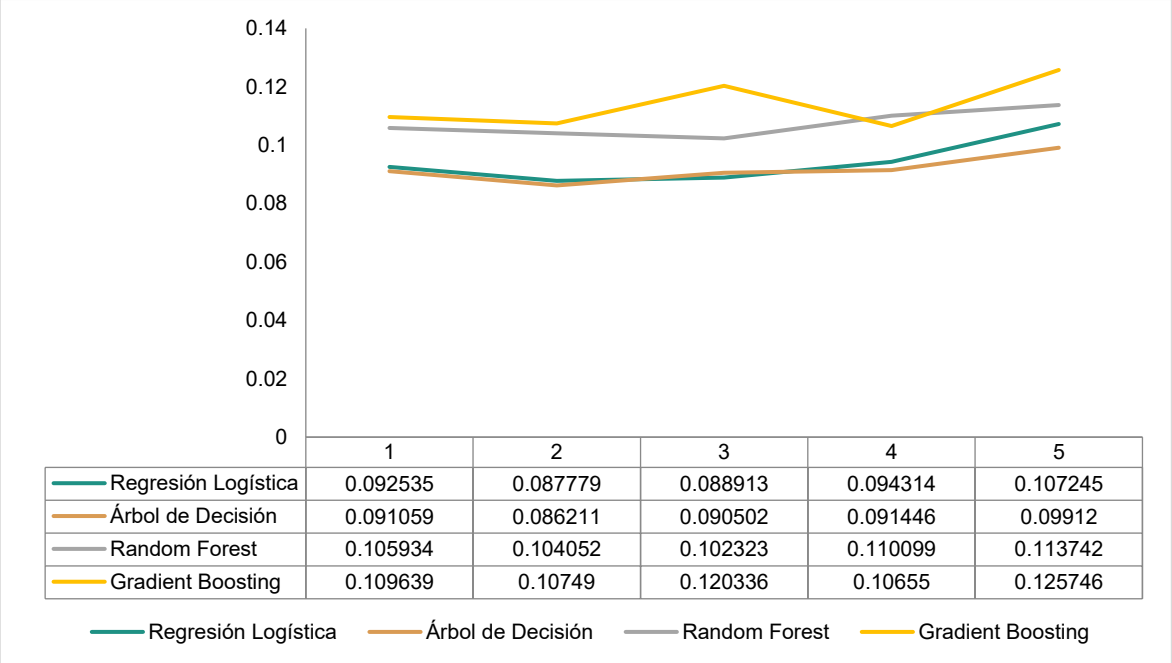
Los resultados de cada partición permitieron examinar el desempeño individual de los modelos y determinar su rendimiento promedio mediante Average Precision (AP), métrica utilizada como criterio principal de selección.

Tabla 9. Average Precision obtenido por modelo en cada fold

Fold	Regresión Logística	Árbol de Decisión	Random Forest	Gradient Boosting
1	0,092535	0,091059	0,105934	0,109639
2	0,087779	0,086211	0,104052	0,107490
3	0,088913	0,090502	0,102323	0,120336
4	0,094314	0,091446	0,110099	0,106550
5	0,107245	0,099120	0,113742	0,125746

Fuente: Elaboración propia a partir de resultados_dashboard.json, secciones comparacion_cv y modelos.cv_desv.

Figura 4. AP de validación por fold



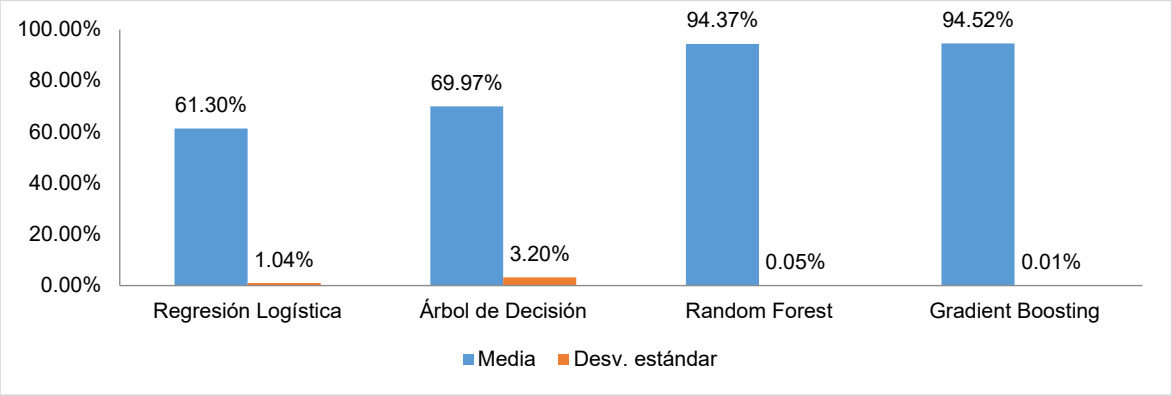
Fuente: Elaboración propia a partir de resultados_dashboard.json, secciones comparacion_cv y modelos.cv_desv.

Tabla 10. Media y desviación estándar de la métrica porcentajes

Modelo	Media	Desv. estándar
Regresión Logística	61,30 %	1,04 %
Árbol de Decisión	69,97 %	3,20 %
Random Forest	94,37 %	0,05 %
Gradient Boosting	94,52 %	0,01 %

Fuente: Elaboración propia a partir de resultados_dashboard.json, secciones comparacion_cv y modelos.cv_desv.

Figura 5. Media y desviación estándar de la métrica porcentajes



Fuente: Elaboración propia a partir de resultados_dashboard.json, secciones comparacion_cv y modelos.cv_desv.

Los resultados de la validación cruzada mostraron que Gradient Boosting obtuvo la mayor media (94,52 %) y la menor desviación estándar (0,01 %), seguido de Random Forest, con una media de 94,37 % y una desviación de 0,05 %. Esto indica que ambos modelos mantuvieron resultados estables entre los cinco folds.

Por otro lado, Regresión Logística alcanzó una media de 61,30 % y una desviación estándar de 1,04 %, mientras que Árbol de Decisión presentó una media de 69,97 % y la

mayor variabilidad (3,20 %). Estos resultados muestran diferencias en la estabilidad de los modelos durante la validación cruzada.

1.4.8 Métricas de evaluación

La evaluación de los cuatro algoritmos mediante validación cruzada estratificada permitió identificar diferencias en su capacidad para clasificar a los solicitantes según el riesgo de incumplimiento crediticio. Para ello, se analizaron seis métricas: Precision, Recall, F1-Score, Accuracy, ROC-AUC y Average Precision (AP).

Los resultados corresponden al promedio de las cinco particiones del conjunto TRAIN. Las métricas Precision, Recall, F1-Score y Accuracy se calcularon utilizando el umbral fijo de clasificación de 0,50, mientras que ROC-AUC y Average Precision permitieron examinar el desempeño de los modelos a través de diferentes umbrales.

Tabla 11. Comparación de las métricas promedio obtenidas mediante validación cruzada

Modelo	Precision	Recall	F1	ROC-AUC	AP	Accuracy
Regresión Logística	0,080081	0,577029	0,140637	0,634474	0,094157	0,612950
Árbol de Decisión	0,090568	0,493146	0,152780	0,630523	0,091668	0,699700
Random Forest	0,175472	0,008657	0,016380	0,654382	0,107230	0,943725
Gradient Boosting	0,000000	0,000000	0,000000	0,668421	0,113952	0,945150

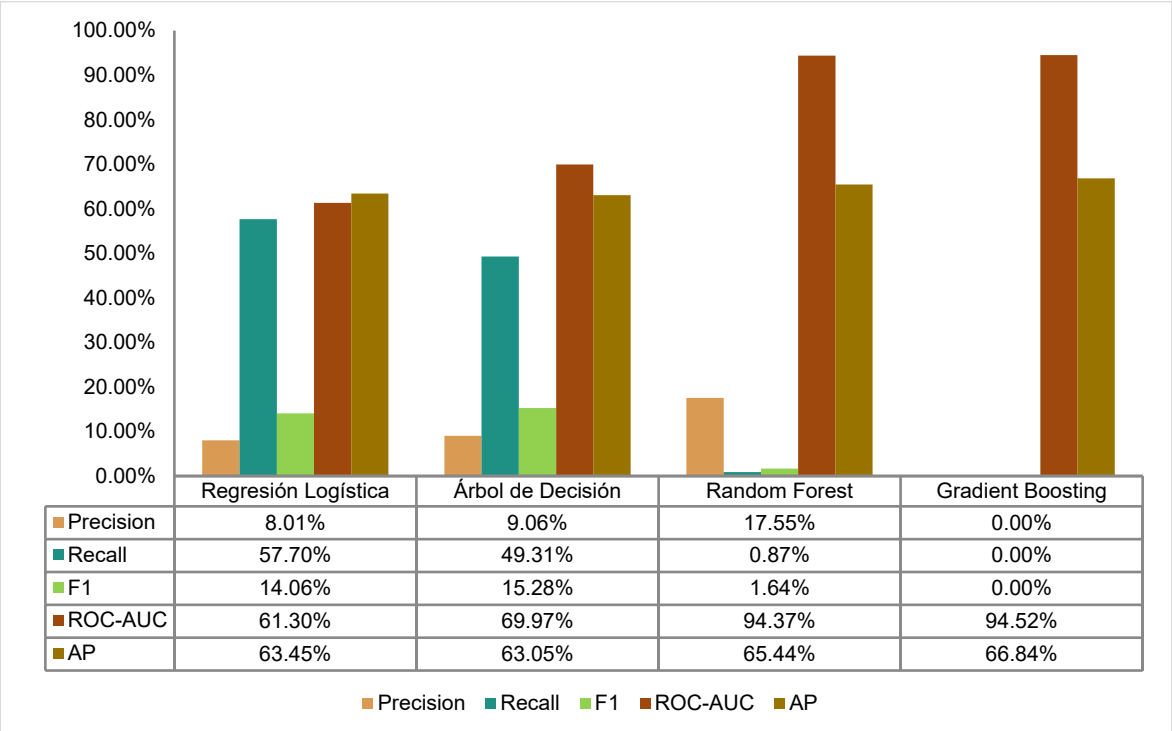
Fuente: Elaboración propia a partir de resultados/comparacion_cv.csv y resultados_dashboard.json, sección comparacion_cv.

Tabla 12. Comparación de las métricas porcentajes

Modelo	Precision	Recall	F1	ROC-AUC	AP	Accuracy
Regresión Logística	8,01 %	57,70 %	14,06 %	61,30 %	63,45 %	9,42 %
Árbol de Decisión	9,06 %	49,31 %	15,28 %	69,97 %	63,05 %	9,17 %
Random Forest	17,55 %	0,87 %	1,64 %	94,37 %	65,44 %	10,72 %
Gradient Boosting	0,00 %	0,00 %	0,00 %	94,52 %	66,84 %	11,40 %

Fuente: Elaboración propia a partir de resultados/comparacion_cv.csv

Figura 6. Comparación de las métricas promedio obtenidas mediante validación cruzada porcentajes



	Regresión Logística	Árbol de Decisión	Random Forest	Gradient Boosting
Precision	8.01%	9.06%	17.55%	0.00%
Recall	57.70%	49.31%	0.87%	0.00%
F1	14.06%	15.28%	1.64%	0.00%
ROC-AUC	61.30%	69.97%	94.37%	94.52%
AP	63.45%	63.05%	65.44%	66.84%

Fuente: Elaboración propia a partir de resultados/comparacion_cv.csv

Gradient Boosting obtuvo los mejores resultados en Average Precision, ROC-AUC y Accuracy, pero no identificó incumplimientos con un umbral de 0,50, por lo que su Recall y

F1-Score fueron cero. En cambio, Regresión Logística alcanzó el mayor Recall (0,577029), el Árbol de Decisión obtuvo el mejor F1-Score y Random Forest registró la mayor Precision promedio. Estos resultados muestran que ningún algoritmo destacó en todas las métricas y que la elección depende de los objetivos del análisis.

1.4.8.1 Matrices de confusión de TRAIN

Las matrices de confusión se construyeron a partir de las predicciones Out-of-Fold (OOF) de los cuatro algoritmos, utilizando un umbral de 0,50. Estas matrices permitieron identificar los verdaderos negativos (VN), falsos positivos (FP), falsos negativos (FN) y verdaderos positivos (VP).

Tabla 13. Resultados de las matrices de confusión OOF

Modelo	VN	FP	FN	VP
Regresión Logística	23.252	14.554	928	1.266
Árbol de Decisión	26.906	10.900	1.112	1.082
Random Forest	37.730	76	2.175	19
Gradient Boosting	37.806	0	2.194	0

Fuente: Elaboración propia a partir de resultados_dashboard.json

La Regresión Logística identificó correctamente 1.266 incumplimientos, aunque también clasificó erróneamente a 14.554 solicitantes como incumplidores. En el caso del Árbol de Decisión, se detectaron 1.082 incumplimientos y se registraron 10.900 falsos positivos.

Random Forest identificó solo 19 incumplimientos y dejó sin detectar otros 2.175. Por su parte, Gradient Boosting clasificó los 40.000 registros como pertenecientes a la Clase 0, con 37.806 verdaderos negativos y 2.194 falsos negativos.

Estos resultados muestran que obtener un Accuracy elevado no garantiza una buena detección de los incumplimientos. En este caso, algunos modelos clasificaron correctamente a la mayoría de los solicitantes, pero tuvieron dificultades para identificar los casos positivos, lo que pone de manifiesto la importancia de analizar las matrices de confusión junto con las demás métricas de evaluación.

1.4.8.2 Curvas ROC y Precision–Recall

Para construir ambas curvas, se utilizaron las probabilidades OOF del conjunto de entrenamiento (TRAIN), lo que permitió analizar el desempeño de los modelos al variar el umbral de clasificación.

Tabla 14. Métricas globales OOF asociadas a las curvas

Modelo	ROC-AUC OOF	AP OOF
Regresión Logística	0,634313	0,092071
Árbol de Decisión	0,629447	0,092011
Random Forest	0,654328	0,105270
Gradient Boosting	0,667526	0,111115

Fuente: Elaboración propia a partir de resultados_dashboard.json

El Gradient Boosting obtuvo el mayor valor de ROC-AUC OOF, con 0,667526, seguido de Random Forest. Este resultado indica que Gradient Boosting presentó una mejor

capacidad para distinguir entre las dos clases según esta métrica, aunque su capacidad de discriminación sigue siendo limitada.

La Precision y el Recall según el umbral de clasificación. Gradient Boosting obtuvo el mayor Average Precision (AP) global OOF, con 0,111115, seguido de Random Forest. Ambos superaron la proporción de incumplimientos de TRAIN (0,05485), lo que evidencia su capacidad para identificar casos positivos. Sin embargo, un AP más alto no garantiza un mayor Recall con un umbral de 0,50, por lo que ambas métricas deben analizarse conjuntamente para comparar el desempeño de los modelos.

1.4.9 Selección del modelo ganador

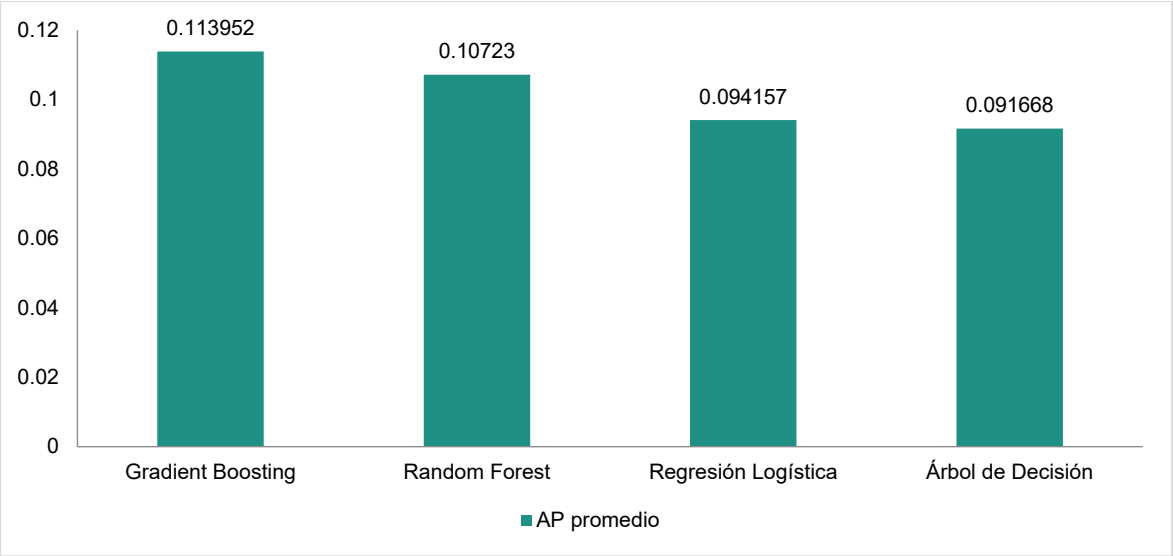
La comparación de los cuatro algoritmos mediante validación cruzada estratificada permitió identificar a Gradient Boosting como el modelo con mayor Average Precision (AP) promedio en el conjunto TRAIN.

Tabla 15. Clasificación de los algoritmos según su AP promedio

Posición	Algoritmo	AP promedio	AP (%)	Recall (%)
1	**Gradient Boosting**	**0,113952**	**11,40 %**	0,00 %
2	Random Forest	0,107230	10,72 %	0,87 %
3	Regresión Logística	0,094157	9,42 %	57,70 %
4	Árbol de Decisión	0,091668	9,17 %	49,31 %

Fuente: Elaboración propia a partir de resultados/comparacion_cv.csv.

Figura 7. Comparación de Average Precision promedio



Fuente: Elaboración propia a partir de resultados/comparacion_cv.csv.

La comparación del Average Precision promedio obtenido por los cuatro algoritmos. Gradient Boosting alcanzó el mayor valor (11,40 %), seguido de Random Forest (10,72 %), Regresión Logística (9,42 %) y Árbol de Decisión (9,17 %).

La diferencia observada entre Gradient Boosting y Random Forest fue de 0,67 puntos porcentuales, mientras que las diferencias respecto a Regresión Logística y Árbol de Decisión fueron de 1,98 y 2,23 puntos porcentuales, respectivamente.

Estos resultados respaldaron la selección de Gradient Boosting conforme al criterio de AP promedio. No obstante, su elección debe interpretarse dentro del alcance experimental, puesto que obtener el mayor Average Precision no garantiza una detección adecuada de incumplimientos con un umbral fijo de clasificación.

1.4.10 Sensibilidad del umbral de clasificación

El análisis de sensibilidad de Gradient Boosting se realizó utilizando las predicciones Out-of-Fold (OOF) de los 40.000 registros de TRAIN. Se evaluaron 19 umbrales entre 0,05 y 0,95 para observar cómo cambiaba la detección de incumplimientos crediticios.

Los resultados mostraron diferencias importantes en la cantidad de incumplimientos identificados y falsos positivos generados, especialmente al utilizar umbrales inferiores a 0,50.

Tabla 16. Variación de Precision y Recall según el umbral de clasificación

Umbral	VN	VP	FP	FN	Precision	Precision (%)	Recall	Recall (%)
0,05	22.792	15.014	788	1.406	0,0856	8,56 %	0,6408	**64,08 %**
0,10	35.049	2.757	1.723	471	0,1459	14,59 %	0,2147	21,47 %
0,15	36.990	816	1.995	199	0,1961	19,61 %	0,0907	9,07 %
0,20	37.488	318	2.109	85	0,2109	21,09 %	0,0387	3,87 %
0,30	37.766	40	2.185	9	0,1837	18,37 %	0,0041	0,41 %
0,50	**37.806**	**0**	**2.194**	**0**	0,0000	**0,00 %**	0,0000	**0,00 %**

Fuente: Elaboración propia a partir de tabla13_umbrales_gb_oof.csv

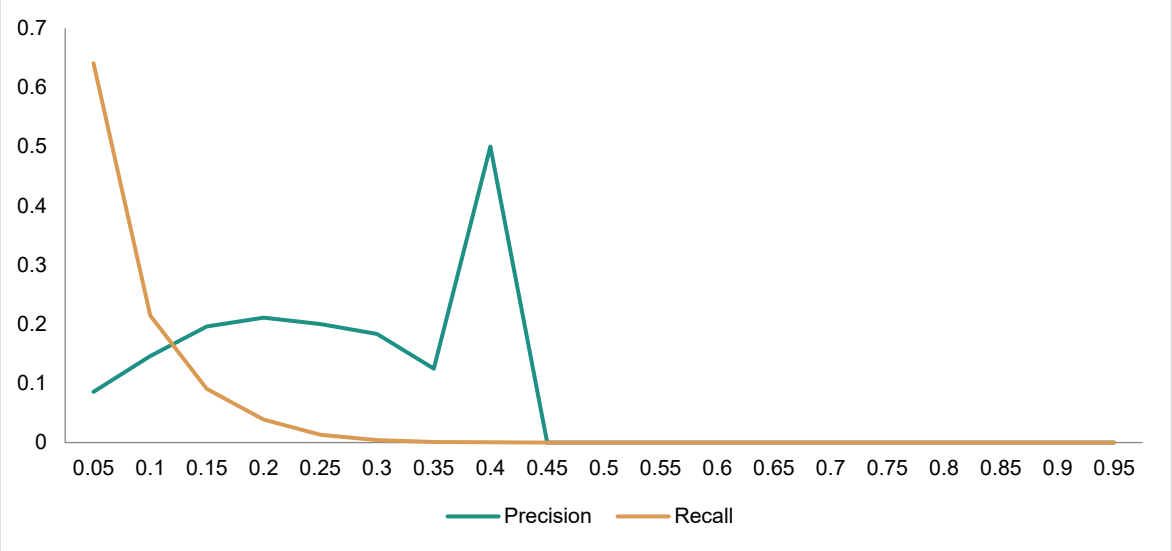
Presenta seis escenarios representativos de los 19 umbrales evaluados, utilizando las predicciones OOF de Gradient Boosting sobre TRAIN.

Con el umbral de 0,50, el modelo no detectó ninguno de los 2.194 incumplimientos reales, obteniendo un Recall del 0,00 %. Al reducirlo a 0,05, identificó 1.406 casos (64,08 %), aunque generó 15.014 falsos positivos.

En los umbrales intermedios disminuyeron tanto los falsos positivos como los incumplimientos detectados. Por ejemplo, con un umbral de 0,15 se identificaron 199 casos y 816 falsos positivos, alcanzando una Precision del 19,61 % y un Recall del 9,07 %.

Estos resultados muestran cómo el umbral influye en la detección de incumplimientos y la generación de falsas alertas. Sin embargo, los escenarios se analizaron únicamente con fines exploratorios, manteniendo el umbral de 0,50 para la evaluación final del modelo

Figura 8. Análisis de sensibilidad del umbral



Fuente: Elaboración propia a partir de resultados_dashboard.json

La gráfica muestra cómo varían Precision y Recall de Gradient Boosting según el umbral de clasificación. El Recall alcanzó su valor más alto (64,08 %) con el umbral de 0,05 y disminuyó conforme este aumentó.

Por su parte, Precision presentó variaciones y alcanzó el 50,00 % con un umbral de 0,40. Sin embargo, este resultado correspondió a un solo verdadero positivo y un falso positivo, por lo que su capacidad de detección fue limitada.

A partir del umbral de 0,45, el modelo dejó de identificar casos positivos, obteniendo Precision y Recall iguales a cero. Estos resultados muestran que reducir el umbral permite detectar más incumplimientos, pero también aumenta las falsas alertas, lo que evidencia las limitaciones de utilizar únicamente el valor de referencia de 0,50.

1.4.11 Evaluación de costos hipotéticos

La evaluación económica se realizó mediante la comparación de escenarios hipotéticos que permitieron analizar el impacto de los errores de clasificación.

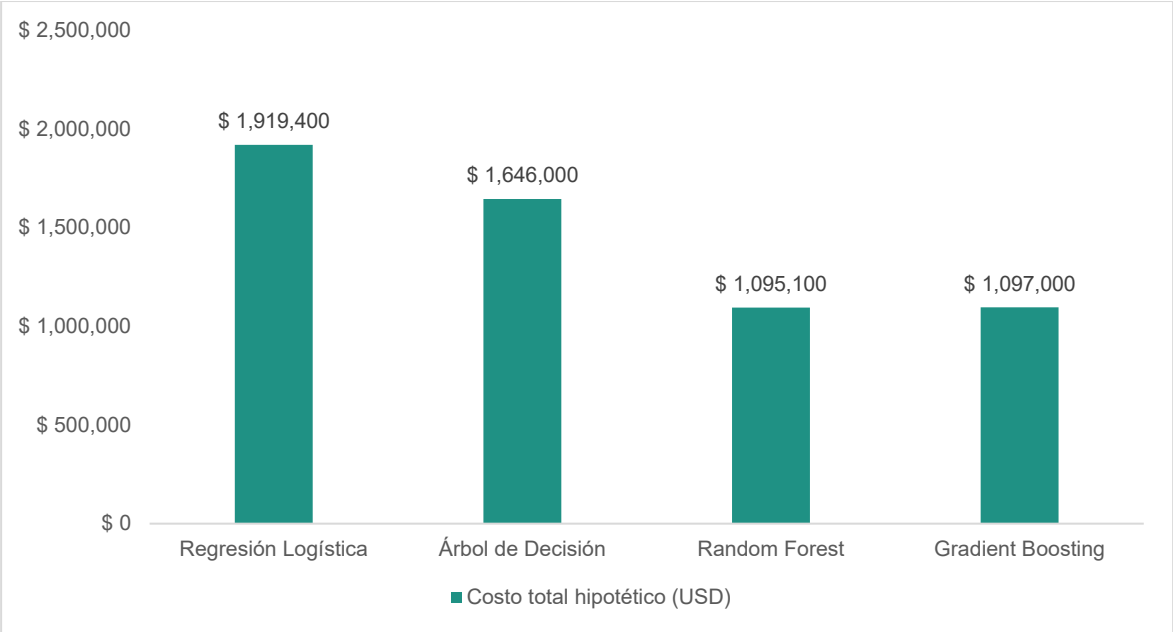
Para este análisis, se asignó un costo de USD 500 a cada falso negativo y de USD 100 a cada falso positivo. Estos valores se establecieron con fines académicos y se utilizaron únicamente para ilustrar las posibles consecuencias económicas de los errores cometidos por los modelos.

Tabla 17. Métricas globales OOF asociadas a las curvas

Modelo	FN	FP	Costo total (USD)	Modelo
Regresión Logística	928	14.554	1.919.400	Regresión Logística
Árbol de Decisión	1.112	10.900	1.646.000	Árbol de Decisión
Random Forest	2.175	76	1.095.100	Random Forest
Gradient Boosting	2.194	0	1.097.000	Gradient Boosting

Fuente: Elaboración propia a partir de resultados/escenarios_costo_oof.csv, considerando umbral 0,50.

Figura 9. Comparación de costos hipotéticos por modelo



Fuente: Elaboración propia a partir de resultados/escenarios_costo_oof.csv, considerando umbral 0,50.

Con un umbral de 0,50, Random Forest presentó el menor costo hipotético (USD 1.095.100), seguido de Gradient Boosting (USD 1.097.000), con una diferencia de USD 1.900. Regresión Logística registró el costo más alto, principalmente por sus falsos positivos. En Gradient Boosting, reducir el umbral a 0,15 disminuyó el costo estimado a USD 1.079.100. Estos resultados dependen de los costos asignados a los errores y del umbral utilizado. Al tratarse de una simulación académica, no representan pérdidas reales ni modifican la selección final del modelo, basada en el promedio de Average Precision (AP).

1.4.12 Entrenamiento definitivo y evaluación sobre TEST

Tras seleccionar Gradient Boosting mediante validación cruzada, se entrenó el modelo definitivo con los 40.000 registros de TRAIN y se evaluó una sola vez con los 10.000 registros de TEST, utilizando un umbral de 0,50.

Los resultados mostraron diferencias entre la capacidad del modelo para distinguir los niveles de riesgo y su efectividad para detectar incumplimientos mediante la clasificación binaria.

Tabla 18. Métricas finales de Gradient Boosting en TEST

Métrica	Resultado	Porcentaje
Average Precision	0,124515	12,45 %
ROC-AUC	0,678975	67,90 %
Accuracy	0,945200	94,52 %
Precision	0,000000	0,00 %
Recall	0,000000	0,00 %
F1-Score	0,000000	0,00 %

Fuente: Elaboración propia a partir de resultados/metricas_test_FINAL.csv. resultados/cv

En el conjunto TEST, Gradient Boosting obtuvo un Average Precision (AP) de 0,124515 y un ROC-AUC de 0,678975, lo que indica una capacidad moderada para diferenciar los niveles de riesgo. Sin embargo, con un umbral de 0,50, no identificó ningún incumplimiento, por lo que la Precision, el Recall y el F1-Score fueron de 0 %. Esto evidencia que, pese a su capacidad de discriminación, el modelo presentó limitaciones para detectar casos positivos con el umbral establecido.

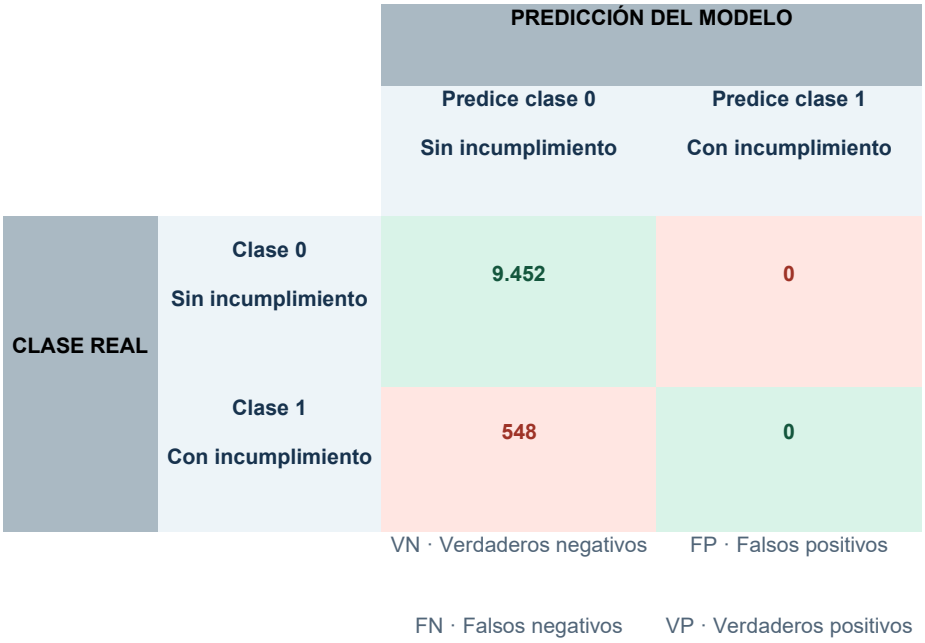
Tabla 19. Matriz de confusión de Gradient Boosting en TEST

Resultado	Cantidad
Verdaderos negativos (VN)	9.452
Falsos positivos (FP)	0
Falsos negativos (FN)	548

Resultado	Cantidad
Verdaderos positivos (VP)	0
Total	10.000

Fuente: Elaboración propia a partir de resultados_dashboard.json

Figura 10. Matriz de confusión final del modelo ganador



Fuente: Elaboración propia a partir de resultados_dashboard.json

La matriz de confusión mostró que Gradient Boosting clasificó correctamente los 9.452 registros de la Clase 0, pero no identificó ninguno de los 548 casos de incumplimiento (Clase 1). Por ello, obtuvo una Accuracy del 94,52 % y un Recall del 0 %. Estos resultados evidencian que, aunque el modelo fue seleccionado por alcanzar el mayor AP promedio en la validación cruzada, con el umbral de 0,50 no logró detectar incumplimientos en el conjunto TEST.

1.4.13 Presentación y visualización del desempeño final

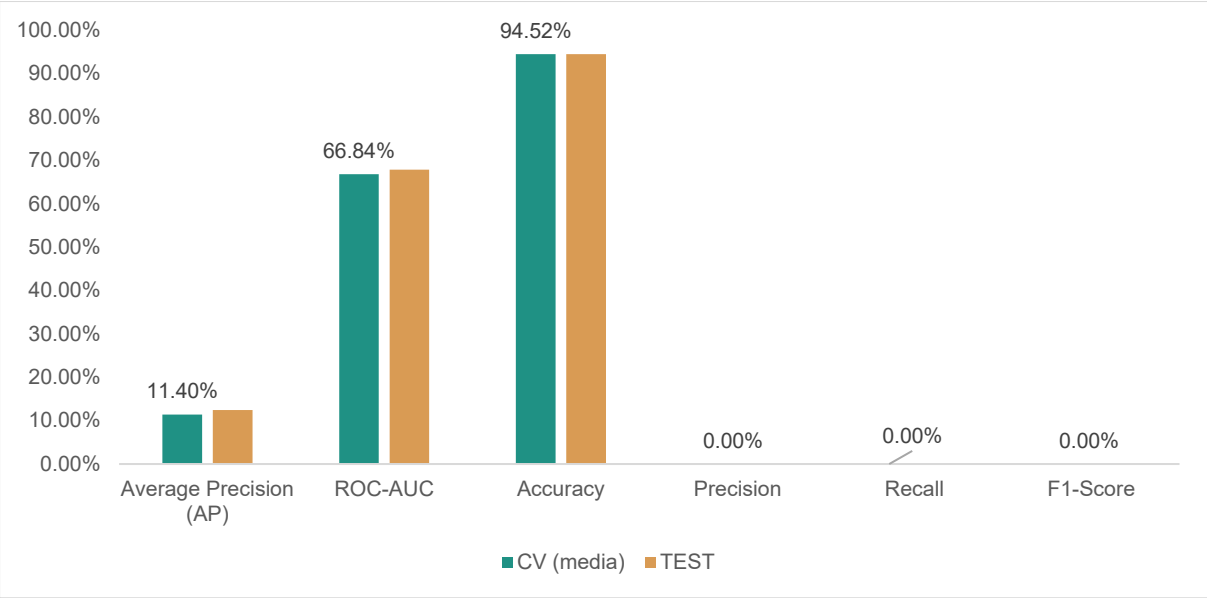
La comparación entre los resultados promedio de la validación cruzada y las métricas obtenidas en TEST permitió analizar la consistencia del modelo seleccionado y observar cómo se comportó frente a datos que no se utilizaron durante su entrenamiento.

Tabla 20. Comparación de resultados CV frente a TEST

Métrica	CV (media)	TEST
Average Precision	0,113952	0,124515
ROC-AUC	0,668421	0,678975
Accuracy	0,945150	0,945200
Precision	0,000000	0,000000
Recall	0,000000	0,000000
F1-Score	0,000000	0,000000

Fuente: Elaboración propia a partir de resultados/cv_vs_test.csv

Figura 11. Comparación CV frente a TEST en Gradient Boosting



Fuente: Elaboración propia a partir de resultados/cv_vs_test.csv

El modelo obtuvo un AP de 0,113952 en validación cruzada y de 0,124515 en TEST. Asimismo, el ROC-AUC aumentó de 0,668421 a 0,678975, mientras que la Accuracy se mantuvo prácticamente estable. Sin embargo, la Precision, el Recall y el F1-Score continuaron en 0 %, lo que indica que, con el umbral de 0,50, el modelo no logró identificar ningún incumplimiento, pese a la mejora en las métricas de discriminación.

1.4.14 Reproducibilidad y consideraciones metodológicas

La implementación permitió mantener la configuración de los algoritmos y documentar los resultados obtenidos en cada etapa del experimento, facilitando el seguimiento y la comparación de su desempeño.

Tabla 21. Configuración reproducible del experimento

Elemento	Configuración
Lenguaje	Python
Semilla aleatoria	42
División TRAIN/TEST	80 % / 20 %
Muestreo	Estratificado
Validación cruzada	5 folds
Criterio de selección	AP promedio
Modelo seleccionado	Gradient Boosting
Umbral fijo	0,50
Evaluación de TEST	Única evaluación del ganador

Fuente: Elaboración propia a partir de src/entrenar.py y resultados_dashboard.json

El uso de una semilla fija permitió reproducir las particiones y mantener condiciones consistentes durante el experimento. Asimismo, mantener TEST separado evitó utilizar los datos de prueba para seleccionar los modelos. No obstante, el desbalance de clases y la incapacidad de Gradient Boosting para detectar incumplimientos con un umbral de 0,50 evidenciaron limitaciones importantes. También se observó un posible sobreajuste en Random Forest, cuyo AP pasó de 0,999407 en entrenamiento a 0,107230 en validación cruzada, lo que refleja una notable disminución del rendimiento al evaluar datos no utilizados directamente para el ajuste del modelo.

En consecuencia, aunque el experimento se estructuró para facilitar su reproducibilidad y mantener una evaluación ordenada, los resultados deben interpretarse teniendo en cuenta el carácter académico y sintético del conjunto de datos utilizado.

1.4.15 Presentación y visualización mediante el dashboard

Los resultados del experimento se integraron en un dashboard interactivo desarrollado con tecnologías web, que permitió consultar y comprender los principales hallazgos. La interfaz organizó la información de las distintas etapas, desde la exploración de los datos y el entrenamiento de los algoritmos hasta la comparación de los modelos y su evaluación final, facilitando la visualización e interpretación de los resultados.

Tabla 22. Correspondencia de las secciones del dashboard con sus fuentes

Sección	Resultados presentados	Fuente principal
Resumen general	Información del dataset y modelo ganador	resultados_dashboard.json
Exploración de datos	Estadísticas y distribuciones	eda_train
Comparación de modelos	Métricas promedio CV	comparacion_cv
Análisis individual	Matrices, curvas y folds	modelos
Análisis de umbrales	Sensibilidad de clasificación	umbrales_oof
Costos hipotéticos	Comparación económica OOF	escenarios_costo_oof.csv
Evaluación TEST	Resultados del ganador	final
Metodología	Procedimiento y parámetros	metodologia
Sección	Resultados presentados	Fuente principal
Resumen general	Información del dataset y modelo ganador	resultados_dashboard.json
Exploración de datos	Estadísticas y distribuciones	eda_train

Fuente: Elaboración propia a partir de src/entrenar.py y resultados_dashboard.json

El dashboard permitió visualizar y comparar los resultados del experimento. Incluyó la distribución de las clases, las características de los solicitantes y el desempeño de los cuatro

algoritmos mediante validación cruzada. También presentó matrices de confusión, curvas ROC y Precision–Recall, resultados por fold y análisis de distintos umbrales.

Finalmente, se mostraron los resultados de Gradient Boosting en TEST, manteniendo la separación entre la selección del modelo con TRAIN y su evaluación independiente. El dashboard no realizó nuevos entrenamientos, sino que presentó los resultados obtenidos previamente en Python, facilitando su interpretación y trazabilidad.

1.5 Discusión de Resultados

Los resultados evidencian diferencias en el desempeño de los cuatro algoritmos evaluados para predecir el incumplimiento crediticio. Gradient Boosting obtuvo el mayor Average Precision (AP) promedio, con 0,113952, y en la evaluación final alcanzó un AP de 0,124515 y un ROC-AUC de 0,678975. Sin embargo, con un umbral de 0,50 no identificó ninguno de los 548 incumplimientos reales. Esto demuestra que un buen rendimiento en ciertas métricas no garantiza la detección de los casos de riesgo, especialmente cuando existe un desbalance de clases, como señalan Saito y Rehmsmeier (2015).

La comparación mostró que la Regresión Logística obtuvo el mayor Recall promedio, aunque generó numerosos falsos positivos; el Árbol de Decisión alcanzó el mejor F1-Score y Random Forest presentó la mayor Precision. Además, el análisis de sensibilidad evidenció que reducir el umbral permite detectar más incumplimientos, pero aumenta las falsas alertas. Por ejemplo, con un umbral de 0,05 se alcanzó un Recall del 64,08 %, a costa de 15.014 falsos positivos. El análisis de costos también mostró que el modelo con mayor AP no necesariamente representa la alternativa más económica.

Por otra parte, Random Forest presentó indicios de sobreajuste, ya que su AP pasó de 0,999407 en entrenamiento a 0,107230 en validación cruzada. Entre las limitaciones del estudio destacan el uso de datos sintéticos, la falta de una optimización exhaustiva de hiperparámetros y la evaluación final únicamente de Gradient Boosting.

En conclusión, Gradient Boosting fue seleccionado por su mayor AP promedio; sin embargo, sus resultados evidencian la necesidad de ajustar los umbrales de decisión y considerar distintas métricas y los costos de los errores para determinar la utilidad real de los modelos de riesgo crediticio. El dashboard interactivo facilitó la comparación y la interpretación de los resultados obtenidos.

1.6 Conclusiones

- Gradient Boosting obtuvo el mejor Average Precision (AP) promedio, con 0,113952, por lo que fue seleccionado como el modelo con mejor desempeño general según el criterio establecido.

- El conjunto de datos presentó un desbalance de clases: el 94,52 % correspondía a solicitantes sin incumplimiento y el 5,48 % a quienes sí incumplieron. Esto evidenció que la exactitud no es suficiente para evaluar la capacidad de detectar casos de riesgo.

- En la evaluación final, Gradient Boosting no identificó ninguno de los 548 incumplimientos reales con un umbral de 0,50. Esto demuestra que un modelo puede obtener buenos resultados en ciertas métricas sin detectar eficazmente los casos de riesgo.

- Reducir el umbral permitió identificar más incumplimientos, aunque también aumentaron los falsos positivos. Por ello, es necesario encontrar un equilibrio entre la

detección de riesgos y los errores de clasificación, considerando además sus costos económicos.

- El dashboard interactivo facilitó la comparación de los algoritmos y la interpretación de sus resultados. Sin embargo, el uso de datos sintéticos y la falta de una optimización exhaustiva limitan la aplicación de los hallazgos a situaciones reales.

- En general, Gradient Boosting fue el modelo mejor valorado según el AP, pero necesita ajustes y pruebas adicionales antes de utilizarse en procesos reales de evaluación del riesgo crediticio.

1.7 Recomedaciones

- Evaluar los modelos con métricas como AP, Recall, Precision y F1-Score, para identificar cuál detecta mejor los incumplimientos crediticios.

- Probar técnicas para equilibrar los datos y optimizar los hiperparámetros, con el fin de mejorar los resultados y evitar errores durante el entrenamiento.

- Ajustar los umbrales de clasificación y analizar los costos de los falsos positivos y falsos negativos para encontrar un equilibrio entre la detección de riesgos y las pérdidas económicas.

- Mejorar el dashboard interactivo para facilitar la comparación de los modelos y la interpretación de sus resultados.

- Realizar nuevas pruebas con datos reales y anonimizados para comprobar la eficacia de los modelos en situaciones financieras reales antes de considerar su implementación.

1.8 Citas Bibliográficas

[1] L. Breiman, "Random forests," Machine Learning, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[2] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

[3] D. J. Hand and W. E. Henley, "Statistical classification methods in consumer credit scoring: A review," Journal of the Royal Statistical Society: Series A (Statistics in Society), vol. 160, no. 3, pp. 523–541, 1997, doi: 10.1111/j.1467-985X.1997.00078.x.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://www.jmlr.org/papers/v12/pedregosa11a.html

[5] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLOS ONE, vol. 10, no. 3, Art. no. e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[6] Scikit-learn developers, "Average precision score," Scikit-learn. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html

[7] Scikit-learn developers, "Common pitfalls and recommended practices," Scikit-learn. [Online]. Available: https://scikit-learn.org/stable/common_pitfalls.html

[8] Scikit-learn developers, "HistGradientBoostingClassifier," Scikit-learn. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html

[9] Scikit-learn developers, "StratifiedKFold," Scikit-learn. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html

[10] Scikit-learn developers, "Tuning the decision threshold for class prediction," Scikit-learn. [Online]. Available: https://scikit-learn.org/stable/modules/classification_threshold.html

2 ELEMENTOS COMPLEMENTARIOS

2.1 Anexos

Anexo_A_Descripcion_del_conjunto_de_datos.

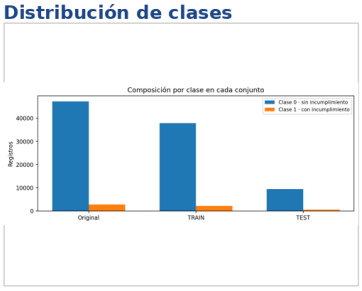
Resumen general

- Registros: 50.000
- Columnas: 16
- Conjunto utilizado: riesgo_crediticio_bigdata.csv

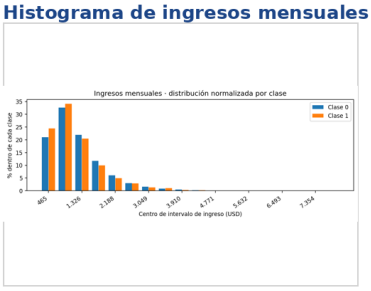
Variable	Tipo	Descripción
applicant_id	object	Identificador único del solicitante
age	int64	Edad del solicitante
monthly_income_usd	float64	monthly_income_usd
employment_years	int64	Antigüedad laboral (años)
employment_type	object	Tipo de empleo
housing_status	object	Situación de vivienda
credit_amount_usd	float64	credit_amount_usd
term_months	int64	term_months
debt_to_income_ratio	float64	debt_to_income_ratio
credit_utilization	float64	Utilización crediticia
late_payments_12m	int64	Atrasos en 12 meses
open_credits	int64	open_credits
savings_usd	float64	savings_usd
loan_purpose	object	Finalidad del préstamo
has_guarantor	int64	Tiene garante
default_12m	int64	Variable objetivo: incumplimiento en 12 meses

Anexo_B_Analisis_exploratorio_de_datos_EDA.

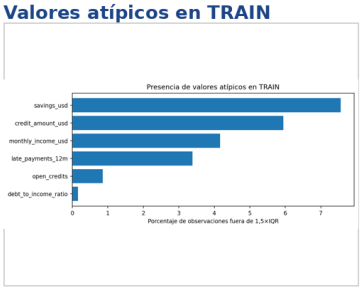
Distribución de clases



Histograma de ingresos mensuales



Valores atípicos en TRAIN



La imagen reúne tres evidencias del análisis exploratorio realizado sobre TRAIN: la distribución de clases, la distribución de ingresos mensuales y la detección de valores atípicos mediante el criterio IQR.

Anexo_C_Implementacion_de_los_modelos_de_clasificacion.

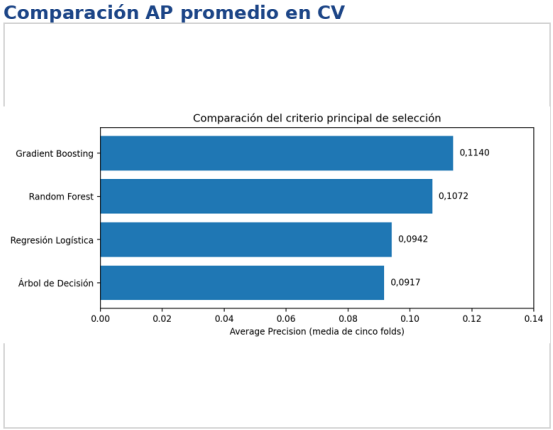
Resumen de los cuatro algoritmos implementados en Python con Scikit-learn y de la configuración general utilizada en el experimento.

Modelo	Parámetros del código	
Regresión Logística	max_iter=1300; class_weight=balanced	
Árbol de Decisión	max_depth=6; min_samples_leaf=25; class_weight=balanced	
Random Forest	n_estimators=110; min_samples_leaf=5; max_features=sqrt; class_weight=	
Gradient Boosting	max_iter=110; max_leaf_nodes=15; learning_rate=0.07; l2_regularization	

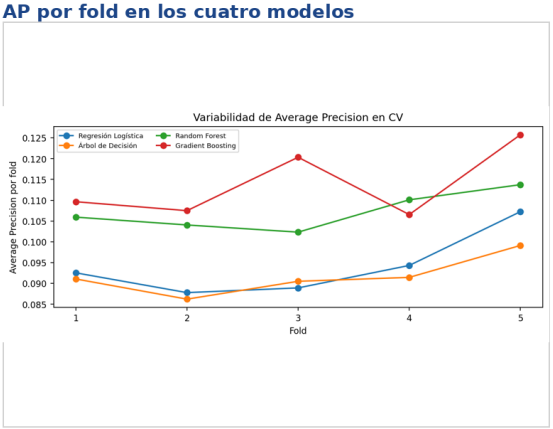
Bibliotecas principales: Pandas, NumPy y Scikit-learn. Los modelos comparados fueron Regresión Logística, Árbol de Decisión, Random Forest y Gradient Boosting.

Anexo_D_Resultados_de_validacion_cruzada.

Comparación AP promedio en CV



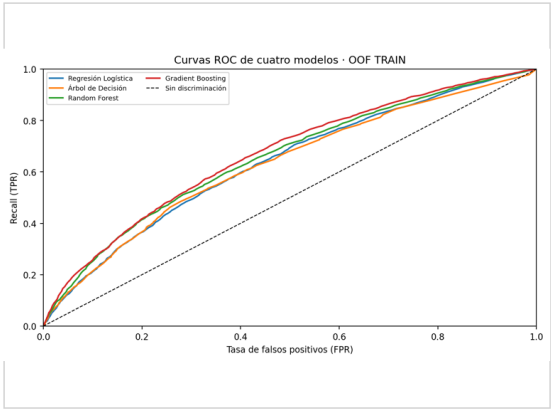
AP por fold en los cuatro modelos



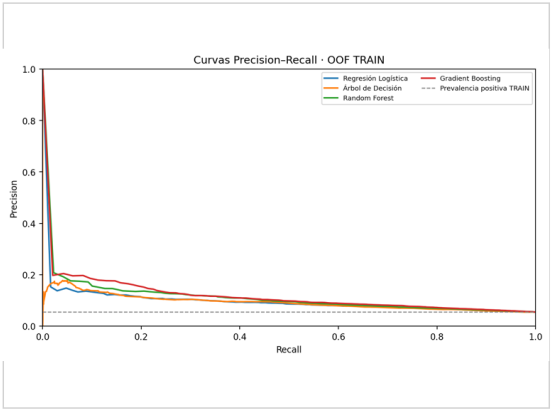
La imagen muestra la comparación global por validación cruzada y el comportamiento del Average Precision en cada una de las cinco particiones de entrenamiento/validación.

Anexo_E_Evaluacion_y_comparacion_de_modelos.

Curvas ROC OOF



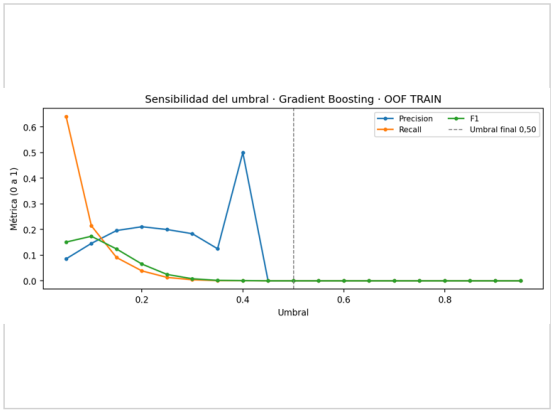
Curvas Precision-Recall OOF



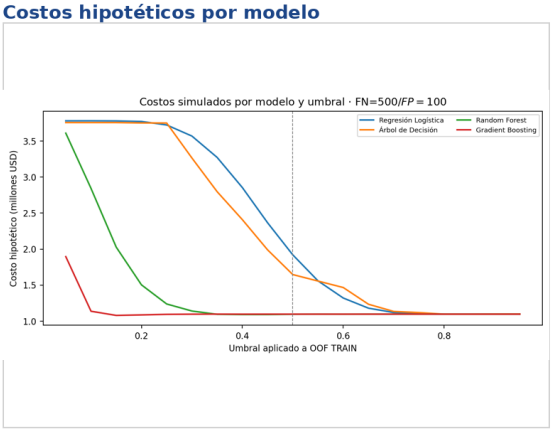
Estas curvas comparan el comportamiento de los cuatro algoritmos sobre predicciones Out-of-Fold (OOF), permitiendo analizar su capacidad de discriminación y la relación entre precisión y sensibilidad.

Anexo_F_Sensibilidad_del_umbral_y_evaluacion_de_costos.

Análisis de sensibilidad del umbral



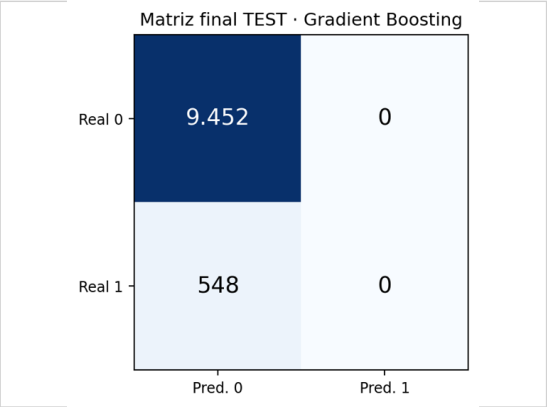
Costos hipotéticos por modelo



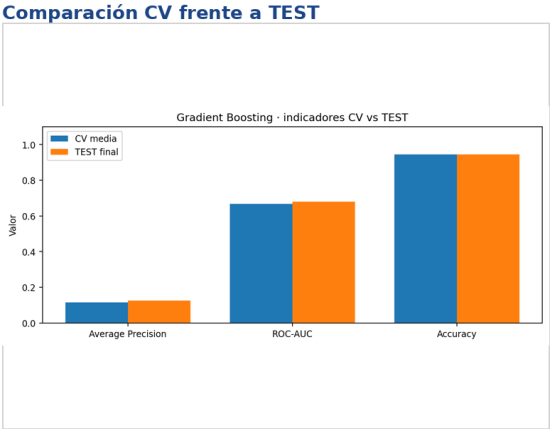
Se presentan los 19 umbrales evaluados para Gradient Boosting y la comparación de escenarios de costos hipotéticos calculados con predicciones OOF de TRAIN.

Anexo_G_Evaluacion_final_de_Gradient_Boosting.

Matriz de confusión en TEST



Comparación CV frente a TEST



La evaluación final muestra el desempeño del modelo ganador sobre el conjunto TEST, incluyendo la matriz de confusión y la comparación entre la validación cruzada y la evaluación final.

Anexo_H_Dashboard_interactivo_y_repositorio_del_proyecto

Enlace del dashboard: https://proanodaniel10.github.io/Caso-1-Riesgo-Crediticio/

Secciones del dashboard: Panorama ejecutivo, Exploración de datos, Comparación CV, Laboratorio de modelos, Costos y errores, Evaluación final y Metodología.